

LEVEL 6 PROJECT DOCUMENTATION

AWS CI/CD Pipeline for Java Web Application Deployment

GitHub + Jenkins + Maven + Docker + Amazon ECR + EC2 + Auto Scaling + ALB + Tomcat

Prepared by	Manimaran
AWS Region	Asia Pacific (Mumbai) — ap-south-1
Environment	Amazon Linux 2023 / EC2 / Docker / Tomcat 10
CI/CD Tool	Jenkins
Source Repository	GitHub
Build Tool	Apache Maven
Container Registry / Load Balancer	Amazon ECR / Application Load Balancer

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives
- 4. Technologies and AWS Services
- 5. Prerequisites
- 6. System Architecture
- 7. Complete CI/CD Workflow
- 8. Project Components
- 9. Complete Implementation Procedure – 38 Screenshot Actions
- 10. Jenkins CI/CD Pipeline
- 11. Jenkins Server Environment
- 12. AWS Infrastructure
- 13. IAM and Amazon ECR Integration
- 14. Application Deployment and Validation
- 15. Application Request Flow
- 16. Rolling Deployment Flow
- 17. Verification Checklist
- 18. Troubleshooting and Operational Notes
- 19. Security Notes
- 20. Key Lessons Learned
- 21. Interview / Viva Explanation
- 22. Conclusion
- Appendix A – Complete CI/CD Flow
- Appendix B – Screenshot Evidence Index

1. Project Overview

This project demonstrates an end-to-end Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Java web application on AWS. The application source code is maintained in GitHub. Jenkins retrieves the source code, builds the application using Apache Maven, produces a WAR file, builds a Docker image, and pushes the image to Amazon Elastic Container Registry (ECR). The deployed application runs on Amazon EC2 instances managed through an Auto Scaling Group. The application is containerized with Docker and served by Tomcat 10 on port 8080. An Application Load Balancer (ALB) provides the public HTTP entry point and forwards requests through a target group to healthy application instances. The deployment stage uses an Auto Scaling instance refresh to roll the application instances. The supplied Jenkins evidence shows the instance refresh reaching 100 percent completion and the pipeline finishing with SUCCESS.

2. Problem Statement

Manual Java application deployment requires repeated source retrieval, compilation, packaging, container creation, image delivery and runtime restart activities. These repeated operations increase deployment effort and can introduce configuration mistakes. This project addresses that workflow by automating source checkout, build, container image creation, ECR publication and rolling deployment through Jenkins and AWS services.

3. Objectives

- Implement an automated CI/CD pipeline for a Java web application.
- Store and manage application source code in GitHub.
- Integrate GitHub with Jenkins using a webhook.
- Build the Java web application with Apache Maven and generate a deployable WAR file.
- Containerize the application using Docker and push the image to Amazon ECR.
- Deploy the application on Amazon EC2 instances managed by an Auto Scaling Group.
- Run the application in a Docker container using Tomcat 10 on port 8080.
- Expose the application through an Application Load Balancer on port 80.
- Perform rolling deployment using an EC2 Auto Scaling instance refresh.
- Verify the application through the ALB DNS endpoint and verify pipeline completion.

4. Technologies and AWS Services

Technology / Service	Purpose / Role
GitHub	Source-code repository and webhook integration
Git	Version control
Jenkins	CI/CD automation and deployment orchestration
Apache Maven	Java build and WAR packaging
Java 17	Java build/runtime environment
Docker	Application containerization
Amazon ECR	Private Docker image registry
Amazon EC2	Compute hosts for Jenkins and application
EC2 Auto Scaling	Application capacity and instance refresh
Application Load Balancer	Public HTTP entry point and traffic distribution
Target Group	Registers application targets and supports health checking
Tomcat 10	Java web application server
IAM	AWS permissions for EC2/Jenkins operations
Amazon VPC	Network isolation and subnet architecture
Amazon Linux 2023	EC2 operating system

5. Prerequisites

- AWS account with permissions for VPC, EC2, ECR, IAM, Auto Scaling and Elastic Load Balancing.
- Amazon Linux 2023 EC2 environment for Jenkins and application hosting.
- Java 17, Maven, Git, Docker and AWS CLI available on the Jenkins server.
- GitHub repository containing src/main, pom.xml, Dockerfile, Jenkinsfile and README.md.
- Security groups and subnet routing configured for the intended Jenkins, ALB and application traffic.
- An IAM instance role suitable for EC2-side ECR operations.
- An Application Load Balancer, target group, launch template and Auto Scaling Group for the application fleet.

6. System Architecture

The architecture begins with a developer pushing application changes to GitHub. GitHub is connected to Jenkins, which performs source checkout, Maven build, Docker image creation and ECR publication. The deployment stage initiates an Auto Scaling Group instance refresh. Refreshed EC2 instances run the Dockerized Tomcat application. Users reach the application through an internet-facing Application Load Balancer, which forwards HTTP requests from port 80 to healthy targets serving Tomcat on port 8080.

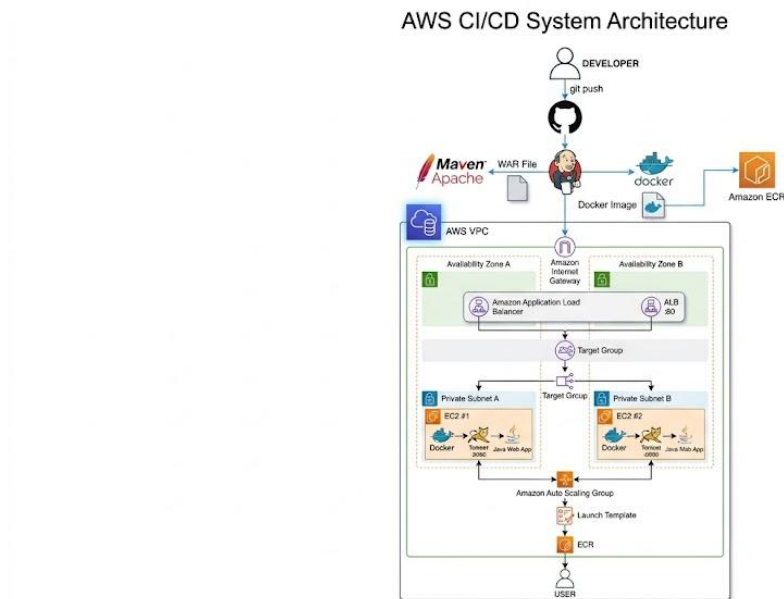


Figure 1. AWS CI/CD System Architecture

7. Complete CI/CD Workflow

Step	Flow	Purpose
1	Developer → GitHub	Application changes are committed and pushed.
2	GitHub → Jenkins	Webhook notifies Jenkins of repository events.
3	Jenkins → Checkout	Jenkins retrieves application source.
4	Jenkins → Maven	Maven compiles/packages the application and creates the WAR.
5	Jenkins → Docker	Docker packages the application into a container image.
6	Jenkins → Amazon ECR	The image is tagged and pushed to the private repository.
7	Jenkins → Auto Scaling Group	Deployment initiates an instance refresh for rolling replacement.
8	EC2 → ECR	Refreshed application instances obtain the required image.
9	EC2 → Tomcat	Docker runs the container and Tomcat serves the application on port 8080.
10	User → ALB → Target Group → EC2	ALB receives HTTP traffic on port 80 and forwards it to healthy targets.

8. Project Components

GitHub Repository: Central source-control location for Java application and CI/CD files.

pom.xml: Maven project/build configuration.

Dockerfile: Docker image build instructions.

Jenkinsfile: Pipeline definition for checkout, build, image creation, ECR push and deployment.

Jenkins EC2: Runs Jenkins and executes CI/CD stages.

Amazon ECR: Private registry storing the Docker image.

Launch Template: Standardizes EC2 instance initialization.

Auto Scaling Group: Maintains application capacity and supports instance refresh.

Target Group: Registers application targets and provides health checks.

Application Load Balancer: Public entry point and traffic distribution.

Tomcat 10 Container: Runs the packaged Java web application on port 8080.

9. Complete Implementation Procedure

The following 38 steps are presented in the chronological order. Each instruction/action is placed before its corresponding image.

Step 1 – Create/verify the VPC

Instruction / Action: Open the Amazon VPC console in ap-south-1 and verify the project VPC exists and is available. This VPC provides the network boundary for the application environment.

Expected Result: The project VPC is visible and available.

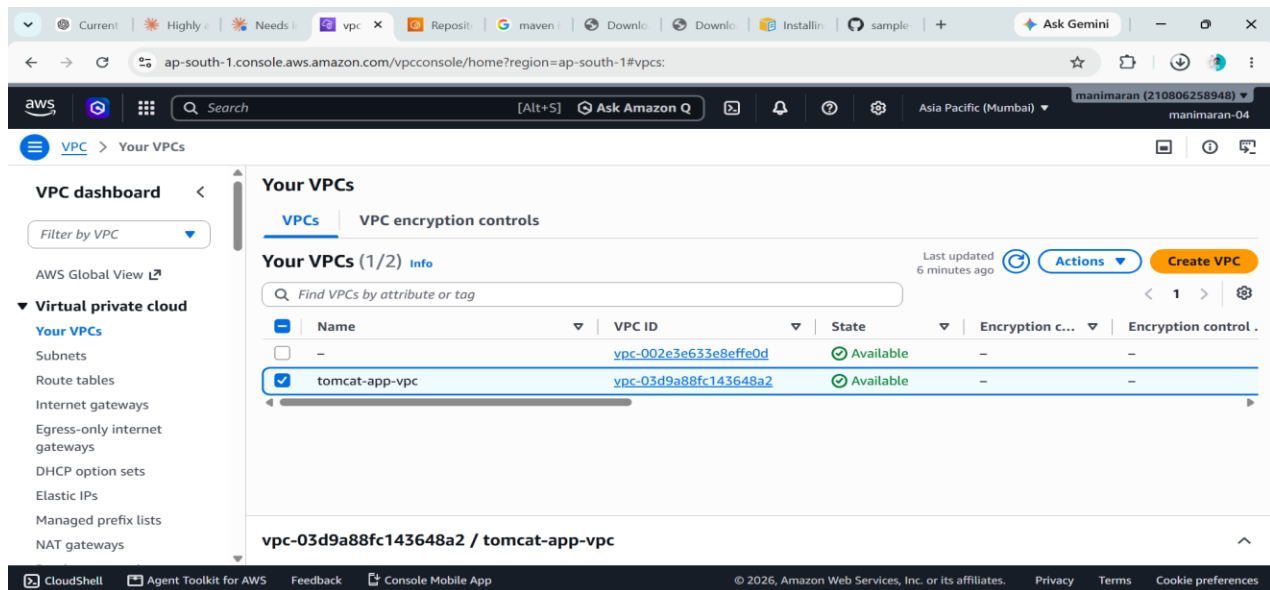


Figure 2. Create/verify the VPC.

Step 2 – Configure the subnets

Instruction / Action: Open the Subnets section and verify the project subnets. Confirm that the required public/private subnet layout is present for the load-balancing and application architecture.

Expected Result: The required subnets are visible and associated with the project VPC.

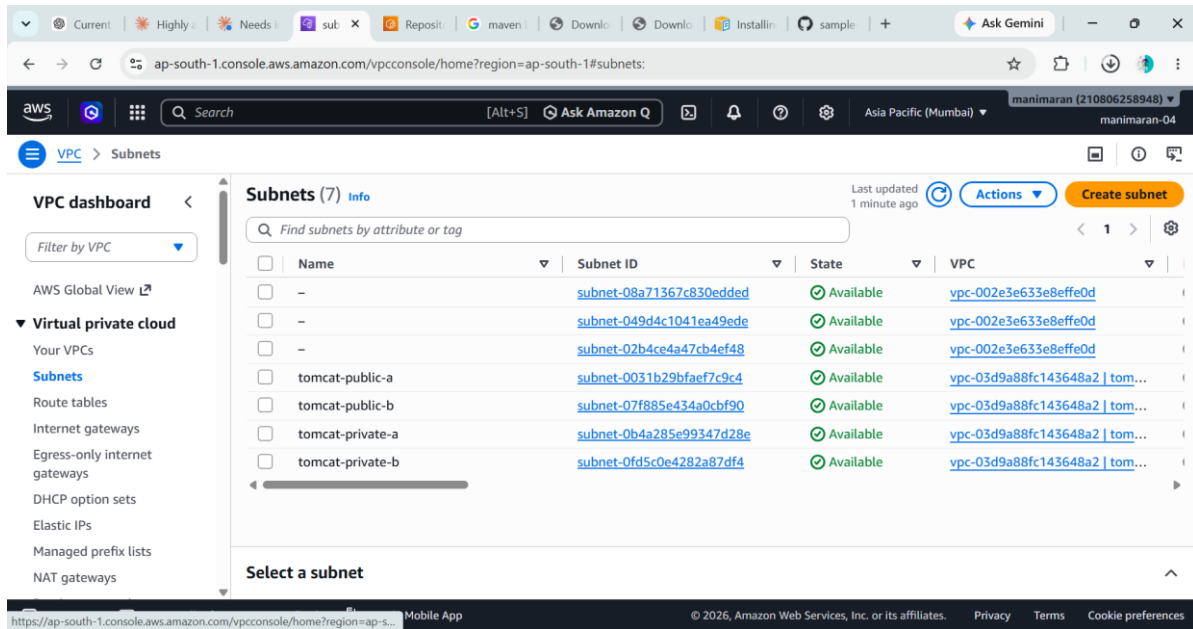


Figure 3. Configure the subnets.

Step 3 – Attach/configure the Internet Gateway

Instruction / Action: Open Internet Gateways and verify the project Internet Gateway is attached to the VPC. This provides Internet connectivity for public-facing resources.

Expected Result: The Internet Gateway is attached to the project VPC.

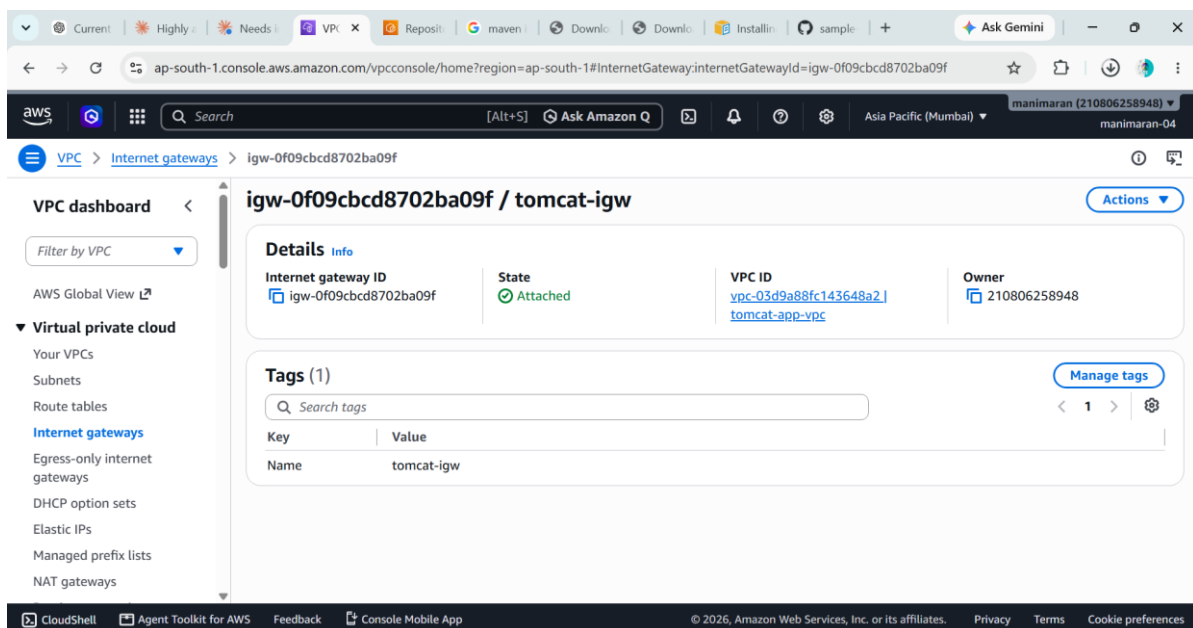


Figure 4. Attach/configure the Internet Gateway.

Step 4 – Create/verify the NAT Gateway

Instruction / Action: Open NAT Gateways and verify the NAT Gateway associated with the project VPC. The NAT path provides outbound Internet connectivity for resources that are not directly Internet-facing.

Expected Result: The NAT Gateway exists for the project VPC.

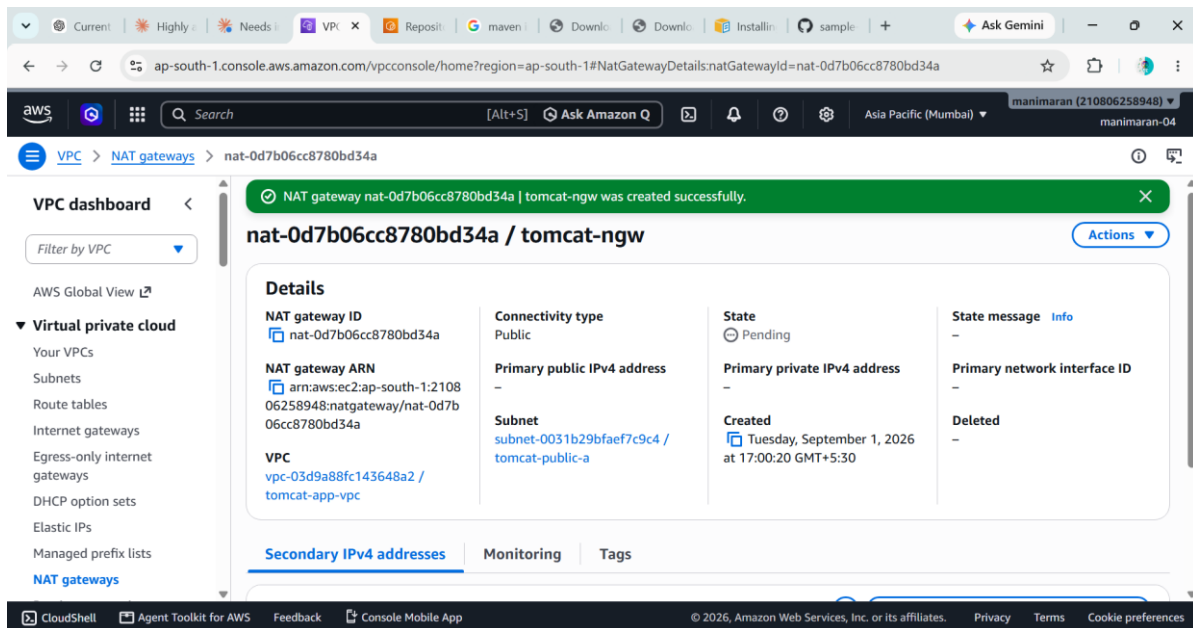


Figure 5. Create/verify the NAT Gateway.

Step 5 – Configure the route tables

Instruction / Action: Open Route Tables and verify the project route tables and their subnet associations. Confirm that routes provide the intended paths through the Internet Gateway and NAT Gateway.

Expected Result: The route tables and subnet associations are visible.

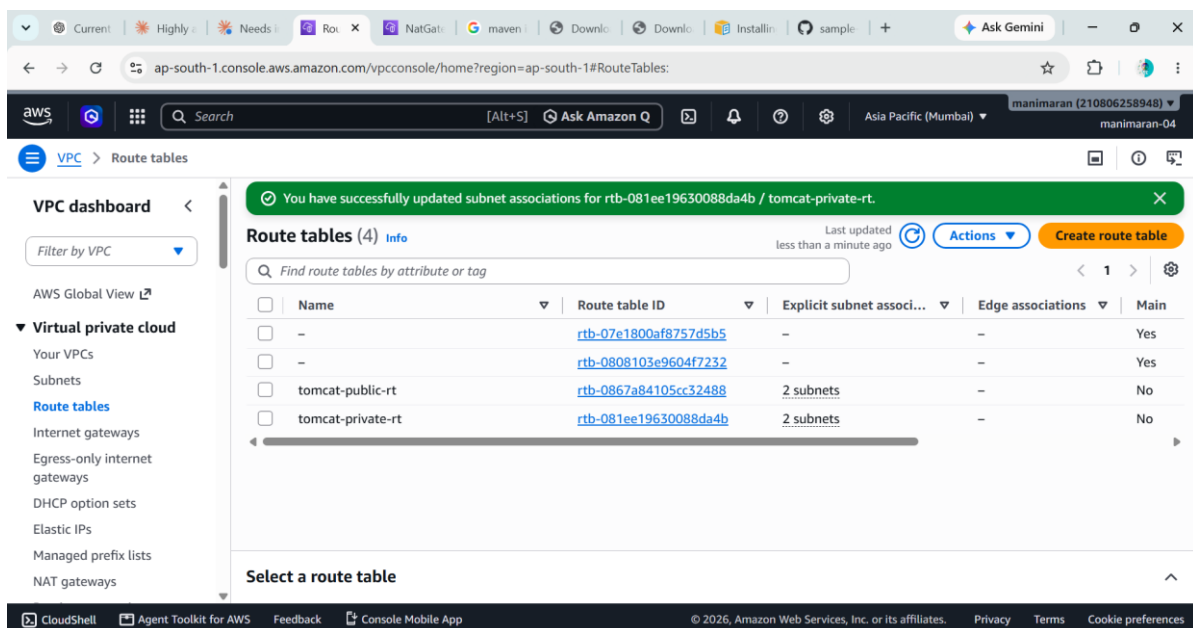


Figure 6. Configure the route tables.

Step 6 – Verify the ECR image/repository

Instruction / Action: Use the AWS CLI to inspect the ECR repository/image information and confirm that the container image exists in the expected repository and region.

Expected Result: The expected ECR image/repository information is returned.

```
latest: digest: sha256:adfe24233636c1bd5347a278cdc2175ee23e6a8582b56db90e3a26541ec3eb32 size: 856
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> aws ecr describe-images --repository-name tomcat-app --region ap-south-1
{
  "imageDetails": [
    {
      "registryId": "210806258948",
      "repositoryName": "tomcat-app",
      "imageDigest": "sha256:f248c3a0e51881cd1fe2ed59e8bb36f4b04261a62855d9783c1b50fc9ee746a5",
      "imageSizeInBytes": 229035219,
      "imagePushedAt": "2026-09-01T18:05:24.649000+05:30",
      "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
      "artifactMediaType": "application/vnd.oci.image.config.v1+json",
      "imageStatus": "ACTIVE"
    },
    {
      "registryId": "210806258948",
      "repositoryName": "tomcat-app",
      "imageDigest": "sha256:362814d81df9715c469c8012fd46118072933671afce0cfa2d43073ac3a4e807",
      "imageSizeInBytes": 1694,
      "imagePushedAt": "2026-09-01T18:05:24.650000+05:30",
      "imageManifestMediaType": "application/vnd.oci.image.manifest.v1+json",
      "artifactMediaType": "application/vnd.oci.image.config.v1+json",
      "imageStatus": "ACTIVE"
    },
    {
      "registryId": "210806258948",
      "repositoryName": "tomcat-app",
      "imageDigest": "sha256:adfe24233636c1bd5347a278cdc2175ee23e6a8582b56db90e3a26541ec3eb32",
      "imageTags": [
```

Figure 7. Verify the ECR image/repository.

Step 7 – Configure the EC2 security group

Instruction / Action: Open EC2 Security Groups and configure/verify the security group used by the ALB/application environment. Allow only the traffic required by the architecture.

Expected Result: The security group is available with the intended rules.

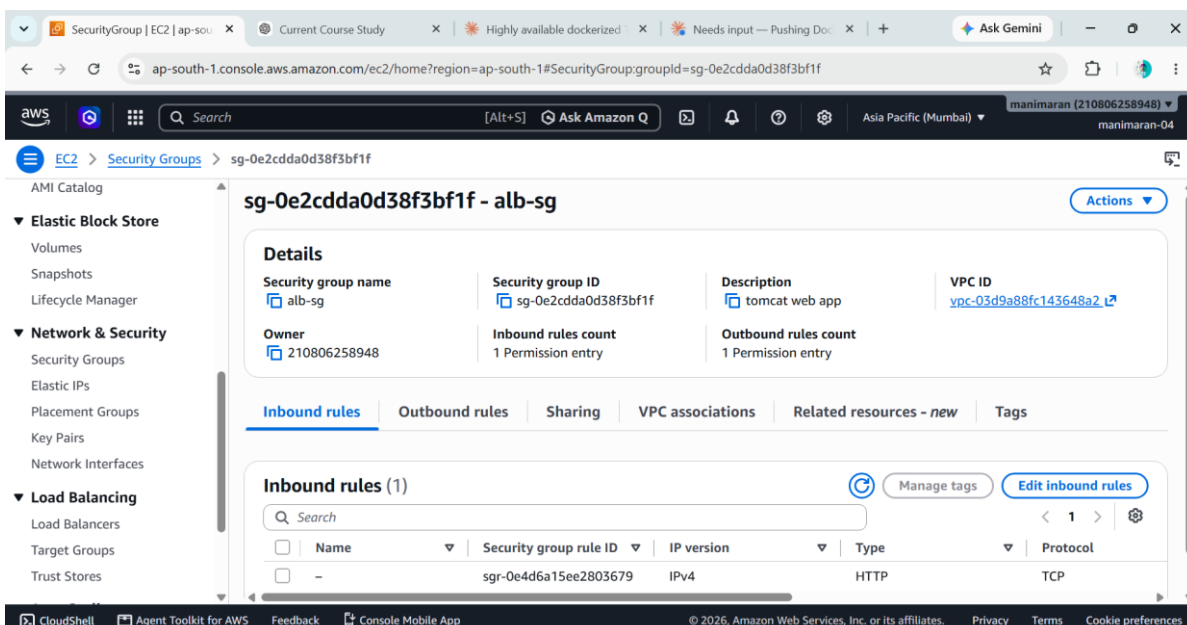


Figure 8. Configure the EC2 security group.

Step 8 – Verify additional application security-group details

Instruction / Action: Open the application security group details and confirm its inbound/outbound rules and VPC association match the intended ALB-to-application traffic flow.

Expected Result: The application security-group configuration is confirmed.

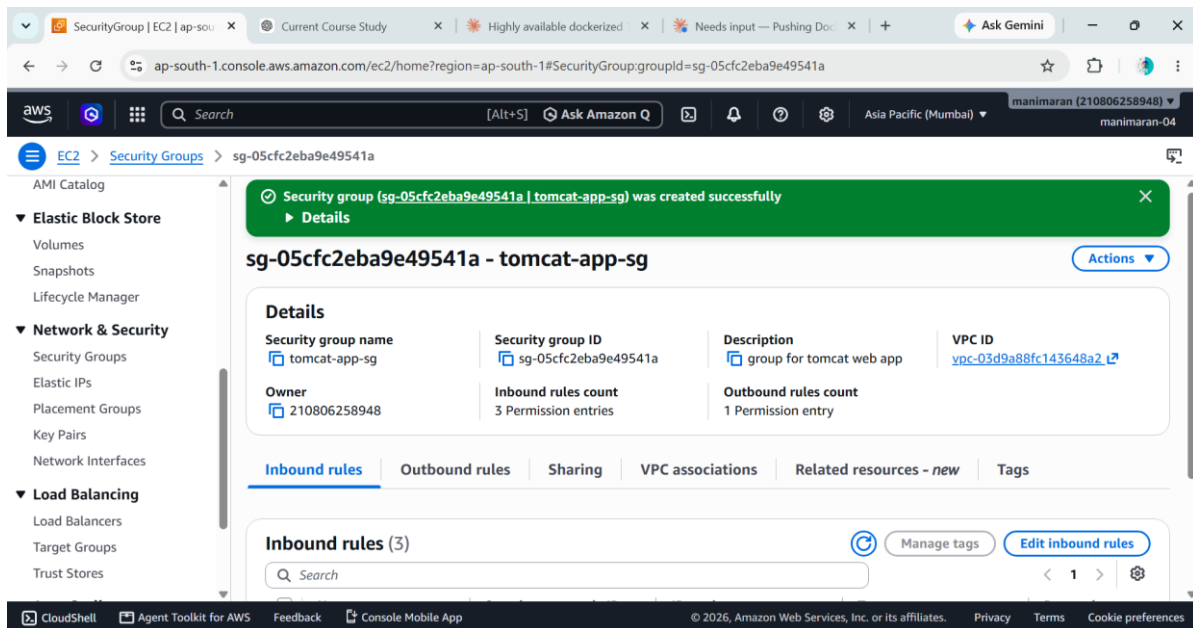


Figure 9. Verify additional application security-group details.

Step 9 – Create/configure the launch template

Instruction / Action: Create the EC2 Launch Template for the application instances. Configure the Amazon Linux 2023 AMI, instance type, security group, storage and user-data initialization required to install/start Docker and run the application.

Expected Result: The launch template is created with the required startup configuration.

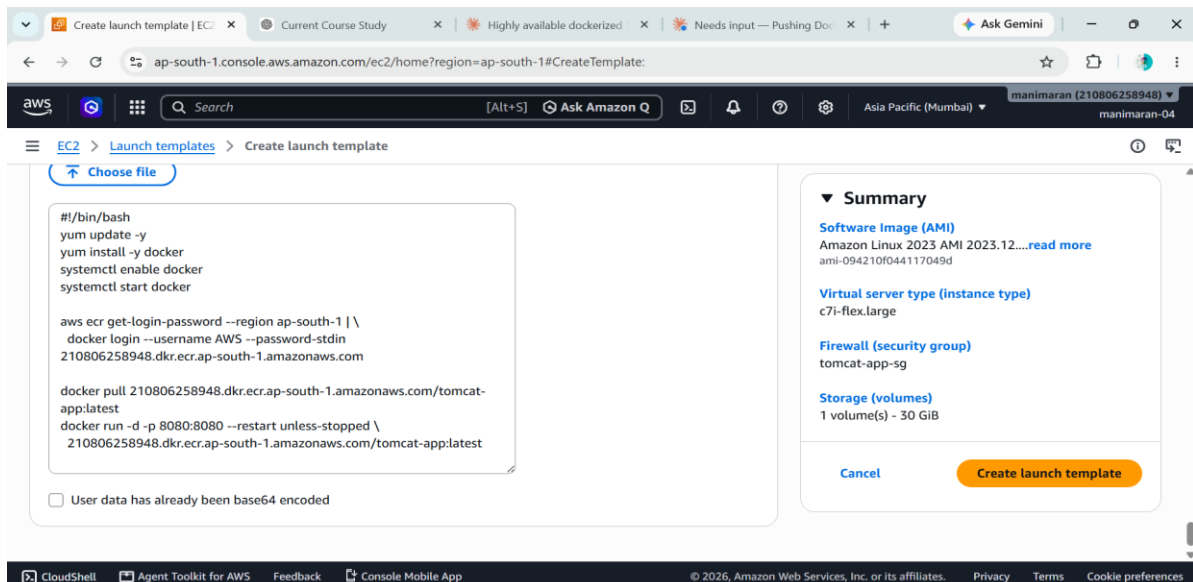


Figure 10. Create/configure the launch template.

Step 10 – Verify the launch template

Instruction / Action: Return to the Launch Templates list and confirm the application launch template is present with the expected version and configuration.

Expected Result: The launch template appears in the EC2 Launch Templates list.

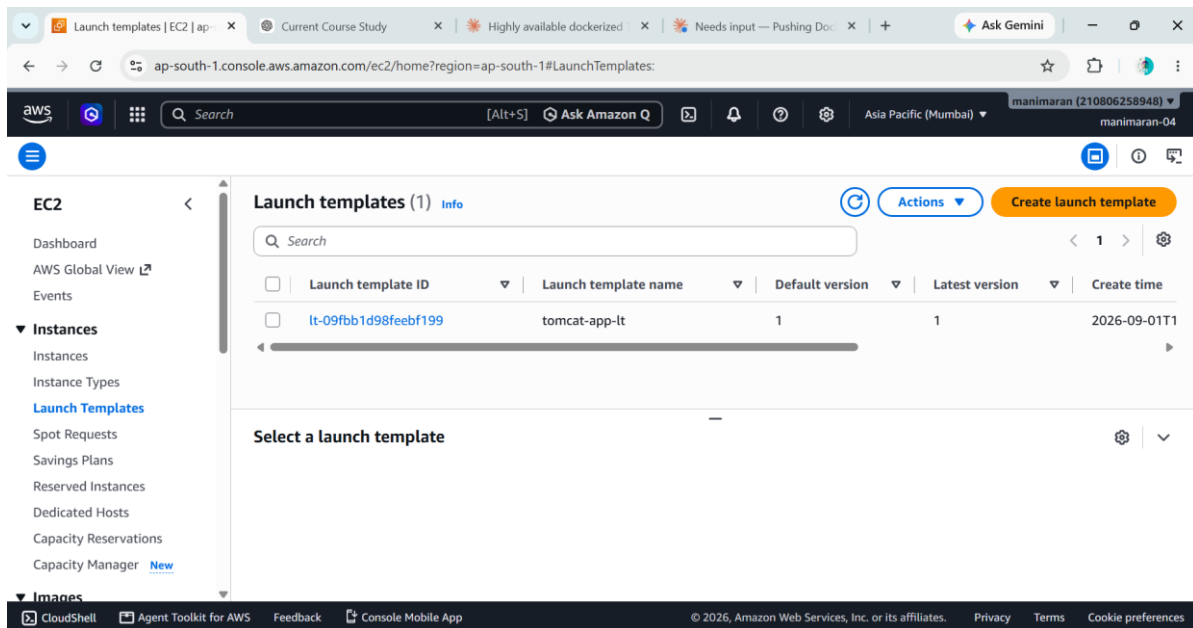


Figure 11. Verify the launch template.

Step 11 – Create/configure the target group

Instruction / Action: Create the target group for the application, use HTTP on port 8080, and configure the health-check path appropriate for the Java/Tomcat application.

Expected Result: The target group is created and ready for application targets.

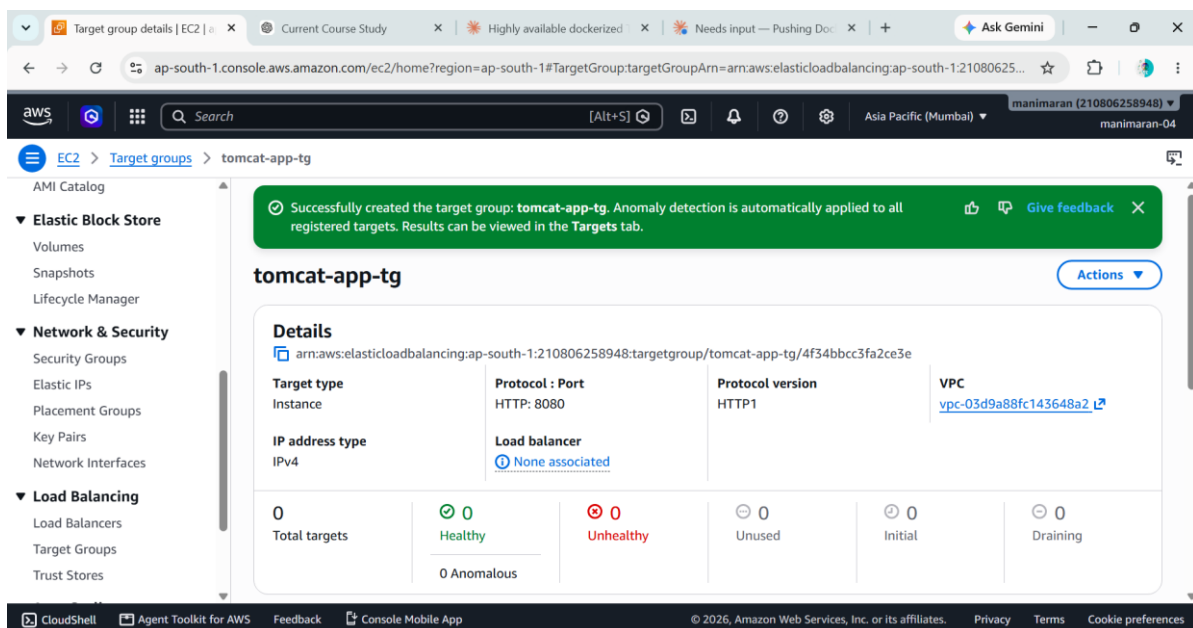


Figure 12. Create/configure the target group.

Step 12 – Create/configure the Application Load Balancer

Instruction / Action: Create an internet-facing Application Load Balancer, associate the required subnets/security group, and configure an HTTP listener on port 80 forwarding to the application target group.

Expected Result: The ALB is active and configured to forward traffic to the target group.

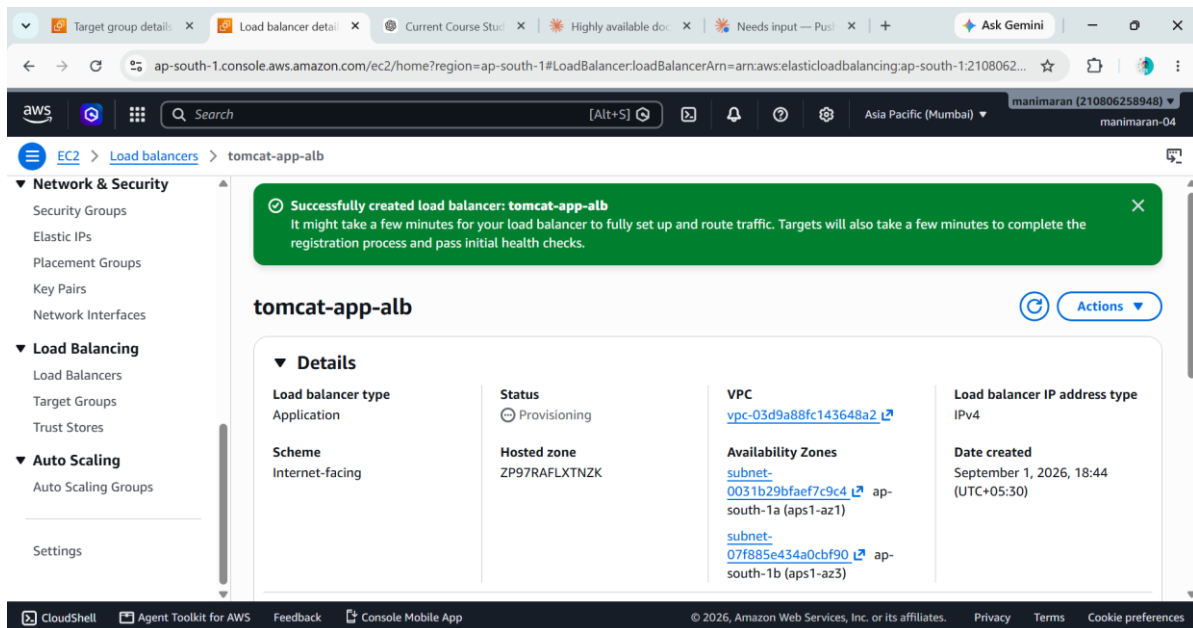


Figure 13. Create/configure the Application Load Balancer.

Step 13 – Start Auto Scaling Group creation

Instruction / Action: Create the Auto Scaling Group using the application launch template. Select the VPC and appropriate availability-zone subnets shown in the implementation.

Expected Result: The Auto Scaling Group creation workflow is started with the intended template.

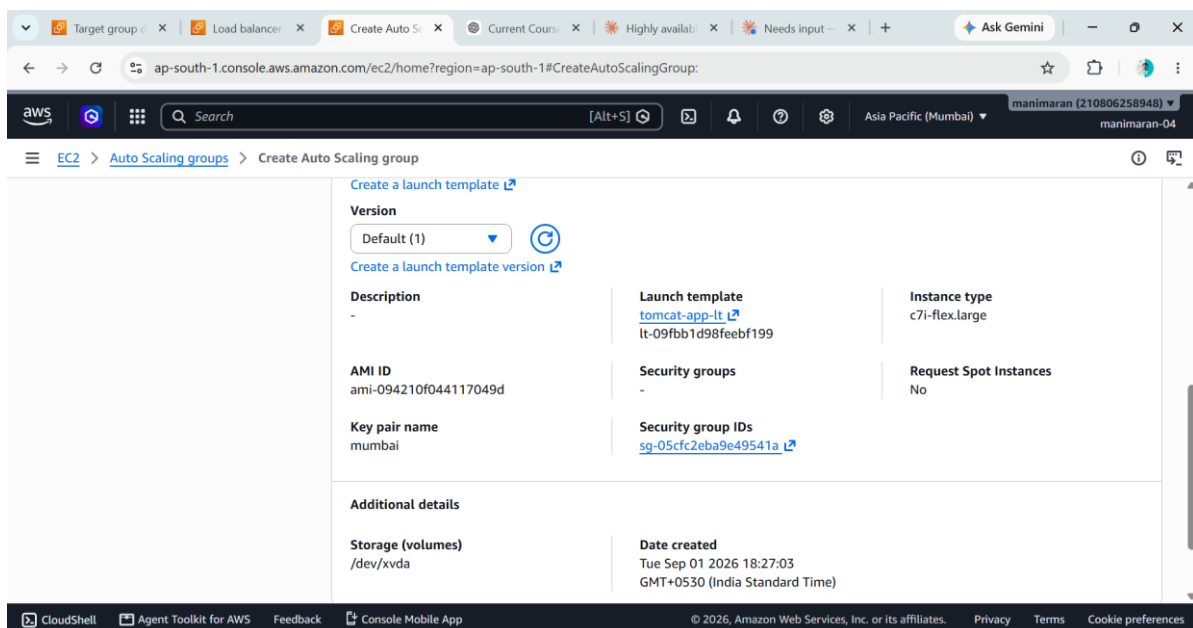


Figure 14. Start Auto Scaling Group creation.

Step 14 – Select availability zones/subnets

Instruction / Action: During Auto Scaling configuration, select the required subnets across the available zones so the application fleet can be distributed across the configured network.

Expected Result: The selected subnets/availability zones are shown in the ASG configuration.

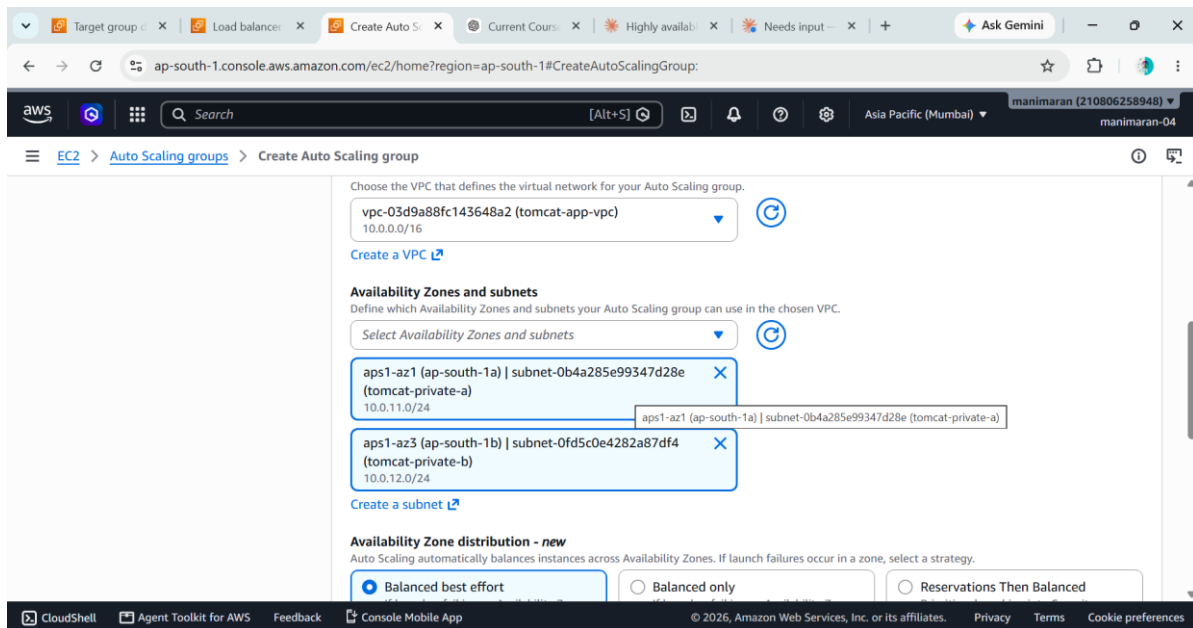


Figure 15. Select availability zones/subnets.

Step 15 – Configure Auto Scaling Group capacity

Instruction / Action: Configure the desired/minimum/maximum capacity and the required load-balancing/health-check settings for the application fleet.

Expected Result: The ASG capacity and load-balancing settings are configured.

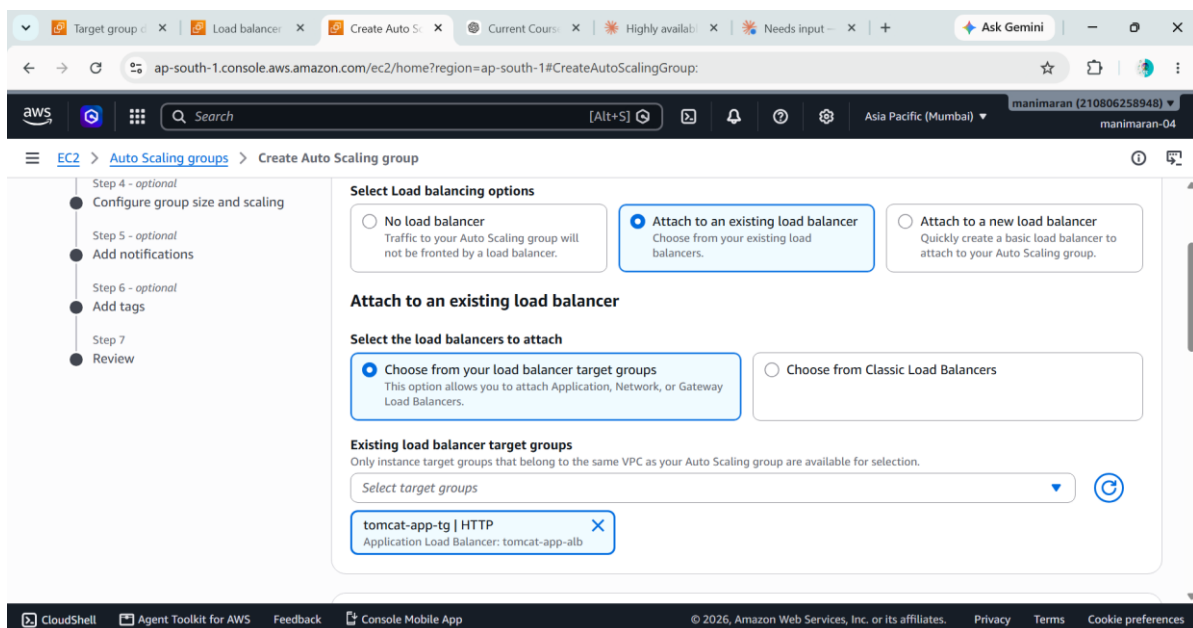


Figure 16. Configure Auto Scaling Group capacity.

Step 16 – Finalize the Auto Scaling Group

Instruction / Action: Complete the Auto Scaling Group creation and verify the final configuration before allowing the group to launch application instances.

Expected Result: The Auto Scaling Group is successfully finalized.

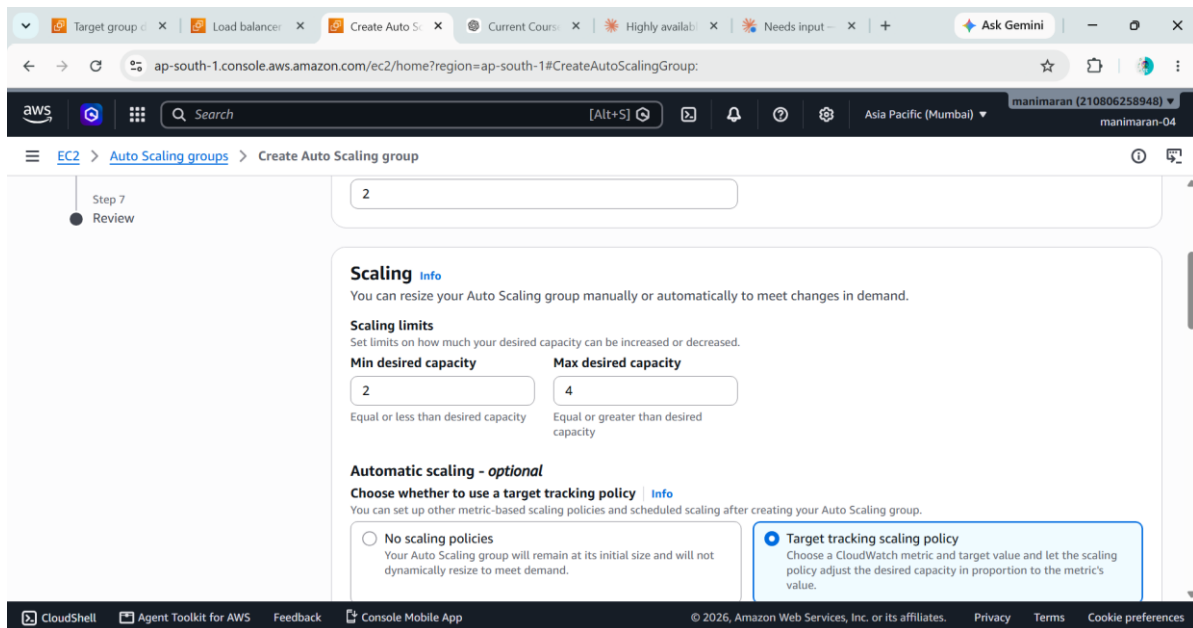


Figure 17. Finalize the Auto Scaling Group.

Step 17 – Verify the Auto Scaling Group

Instruction / Action: Open the Auto Scaling Groups list and confirm the application ASG exists and is associated with the intended launch template and target group.

Expected Result: The application ASG is visible in the ASG list.

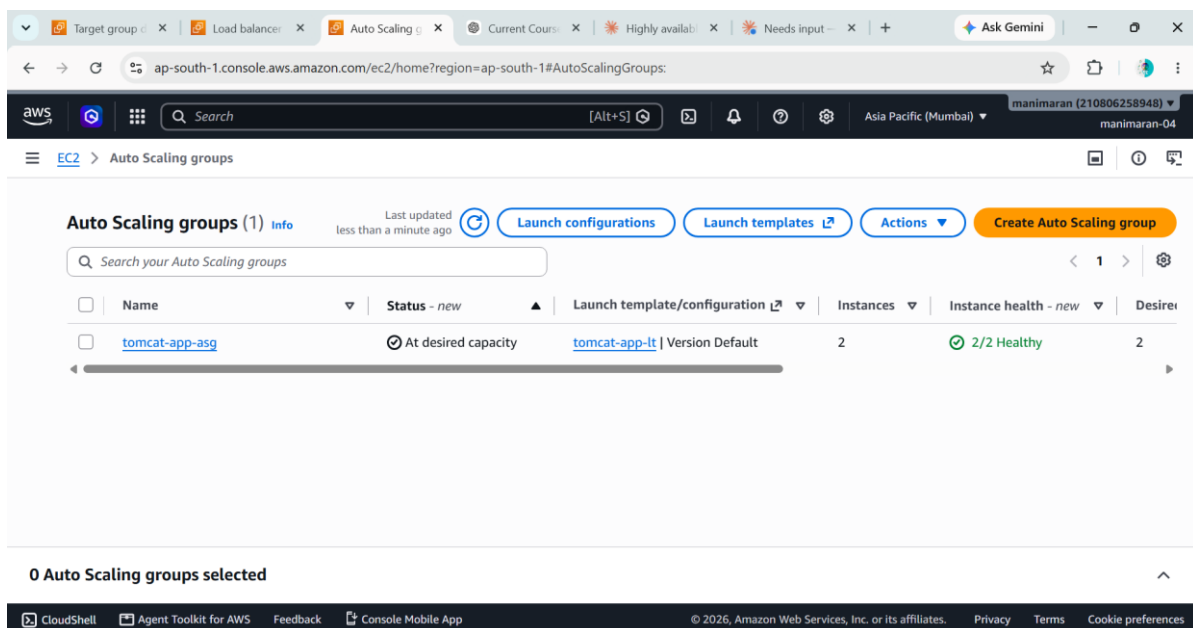


Figure 18. Verify the Auto Scaling Group.

Step 18 – Verify running EC2 instances

Instruction / Action: Open the EC2 Instances list and confirm the application instances launched by the Auto Scaling Group are running and associated with the expected environment.

Expected Result: Running EC2 application instances are visible.

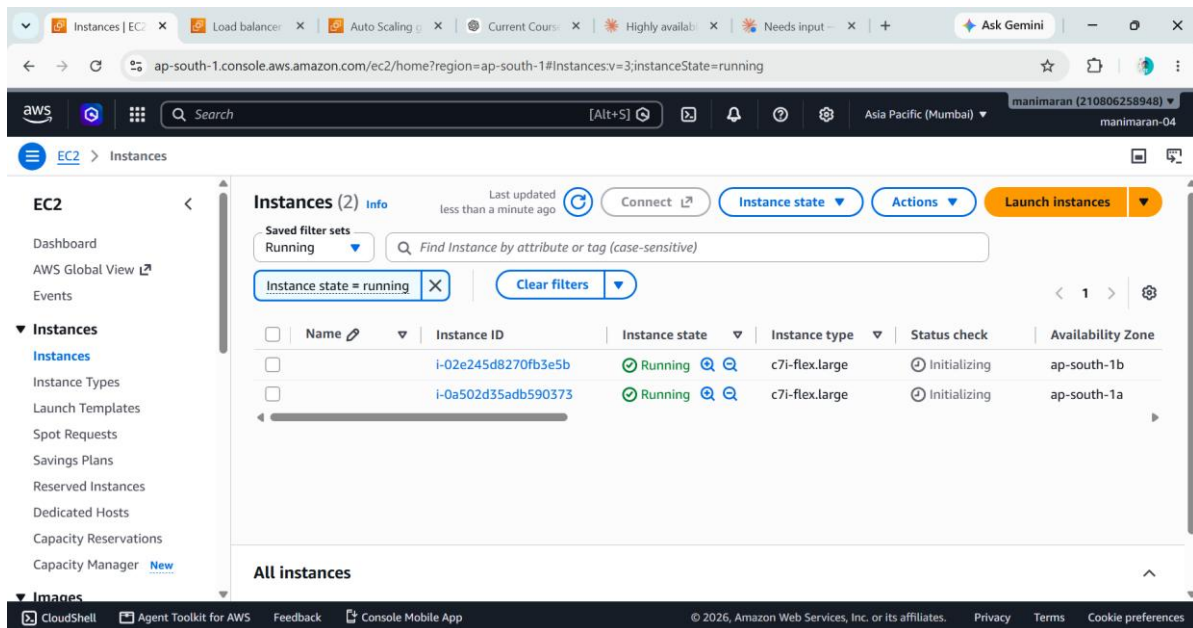


Figure 19. Verify running EC2 instances.

Step 19 – Validate the SampleApp through the ALB

Instruction / Action: Open the ALB DNS endpoint in a browser and verify that the SampleApp home page is served through the load balancer rather than directly through an application-instance address.

Expected Result: The SampleApp Home page is reachable through the ALB.

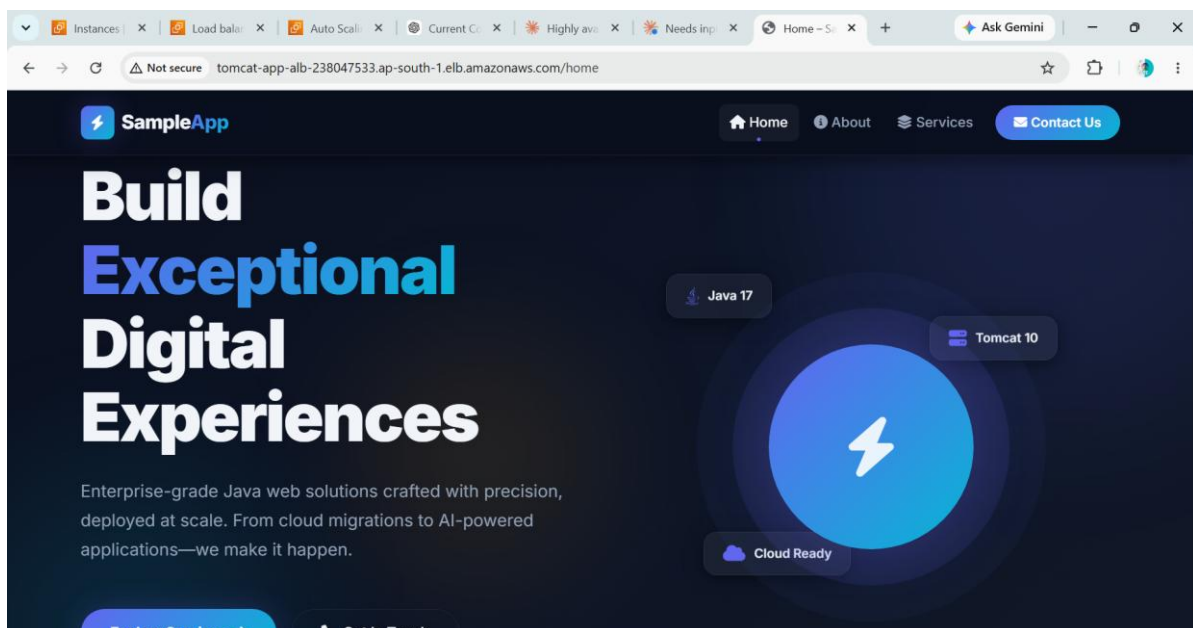


Figure 20. Validate the SampleApp through the ALB.

Step 20 – Review IAM policies

Instruction / Action: Open IAM Policies and verify the policies required for Jenkins/EC2 AWS operations, including permissions needed for ECR and Auto Scaling-related actions.

Expected Result: Required IAM policies are visible for the project.

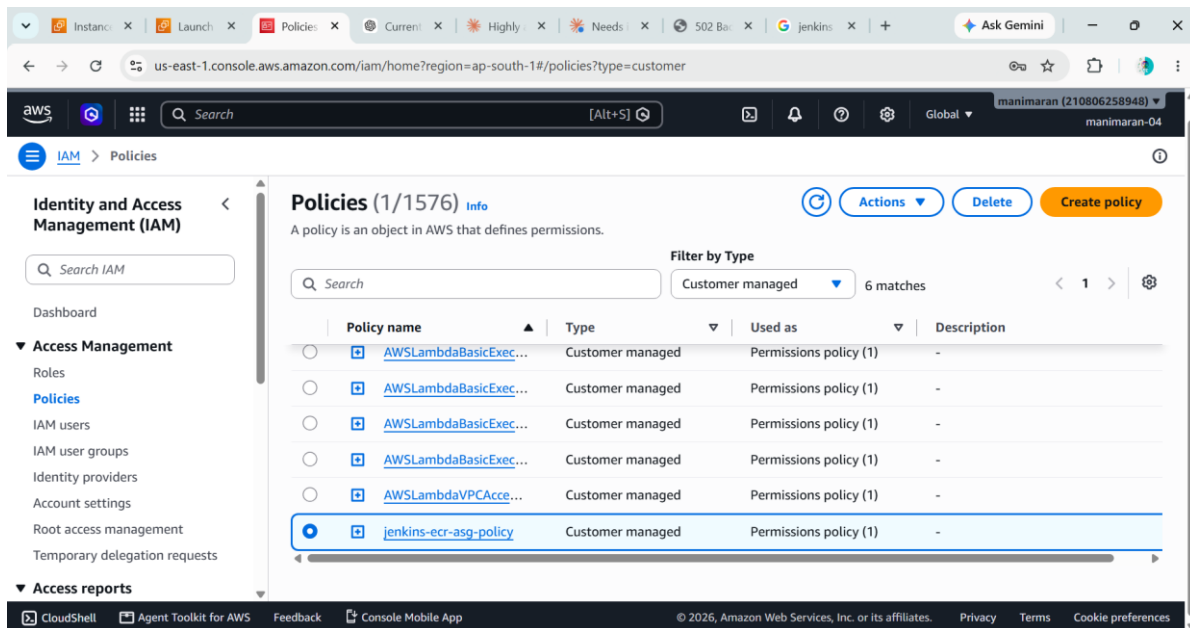


Figure 21. Review IAM policies.

Step 21 – Verify IAM roles

Instruction / Action: Open IAM Roles and verify the project role configuration, including the role used for EC2-side AWS operations. The supplied report identifies the role name as jenkins-ecr-asg-role.

Expected Result: The required IAM role configuration is visible.

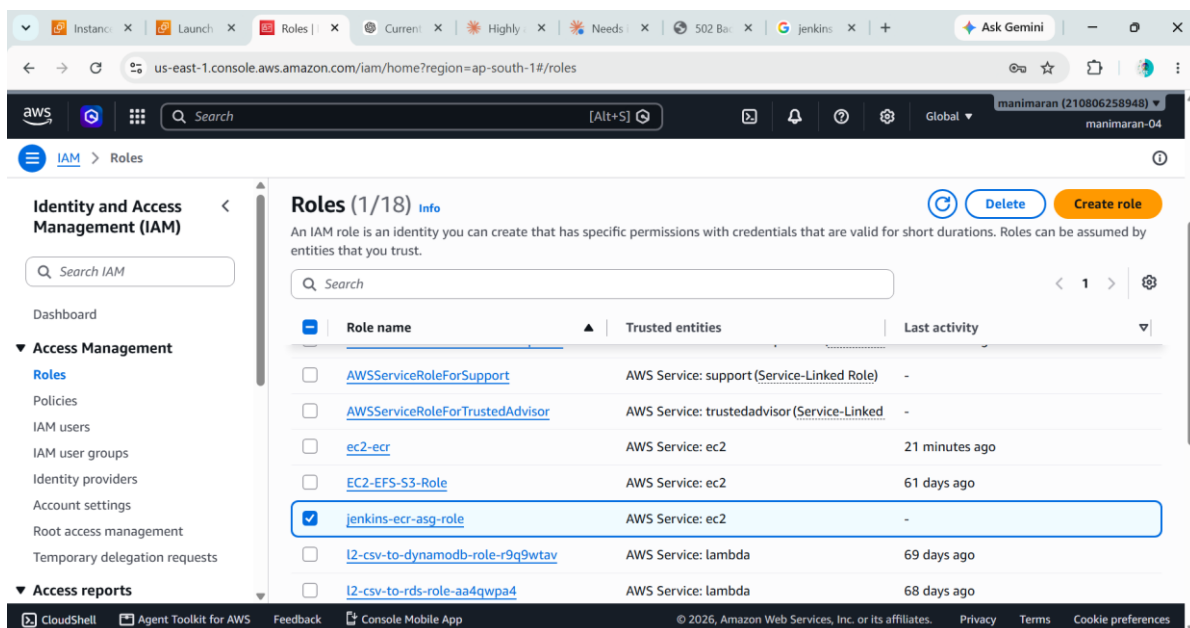


Figure 22. Verify IAM roles.

Step 22 – Verify the EC2 Jenkins/application environment

Instruction / Action: Open the EC2 instance list and identify the Jenkins/application environment instances used by the project.

Expected Result: The Jenkins/application EC2 environment is identifiable.

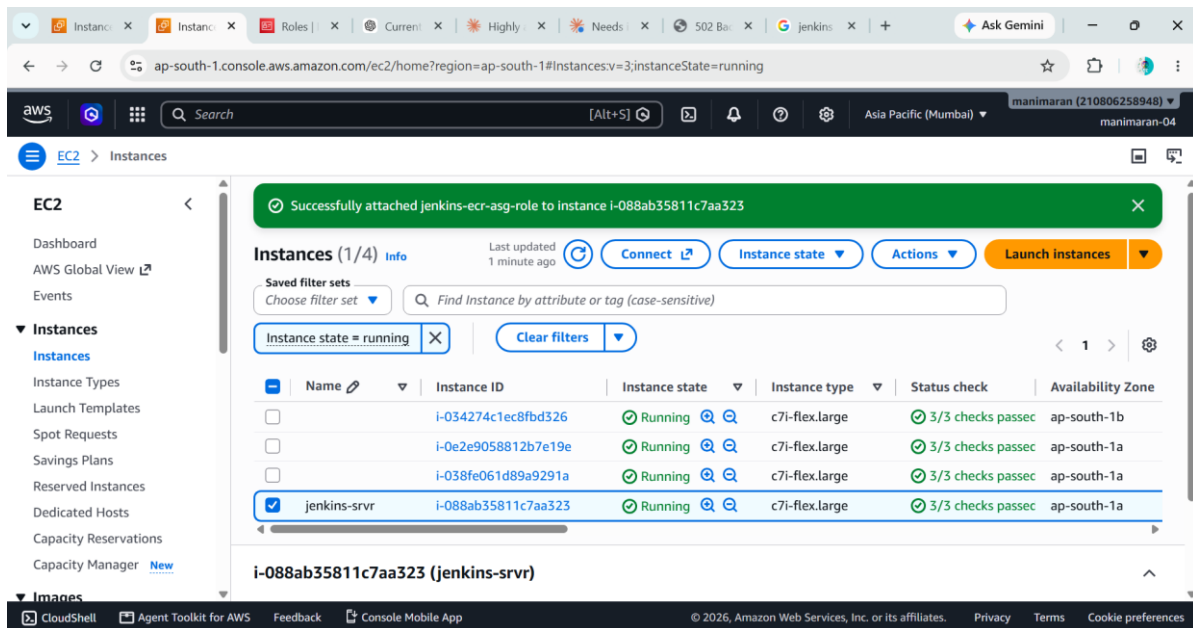


Figure 23. Verify the EC2 Jenkins/application environment.

Step 23 – Verify Jenkins service status

Instruction / Action: Connect to the Jenkins EC2 instance and verify that the Jenkins systemd service is enabled/running. Jenkins must be active before pipeline execution.

Expected Result: Jenkins is active and ready for pipeline execution.

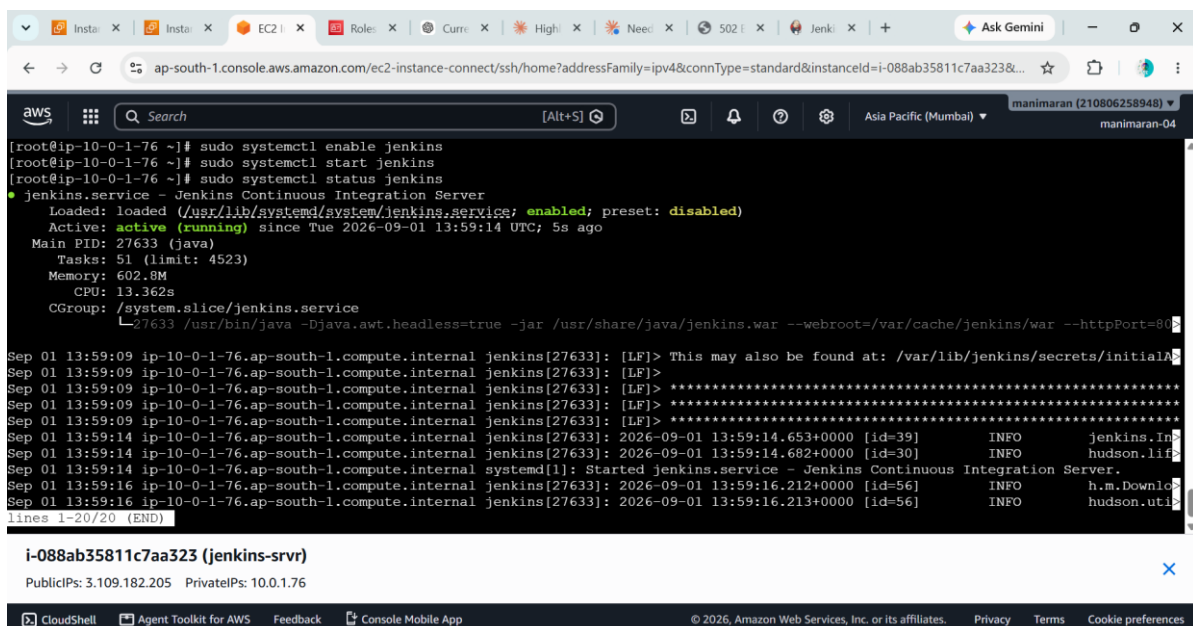
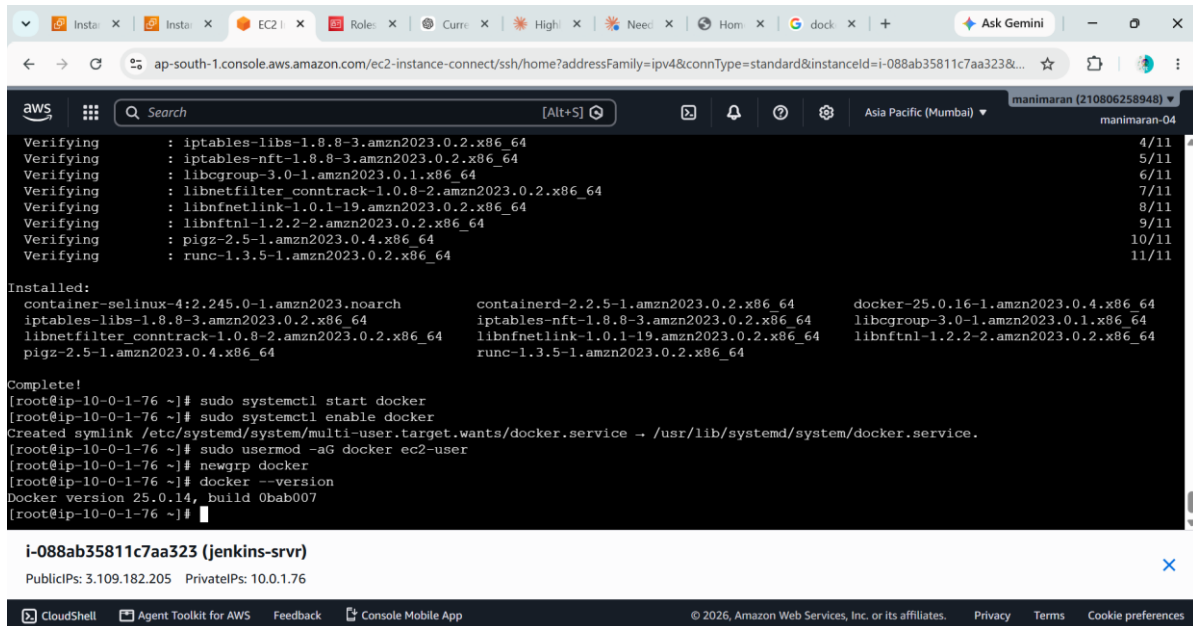


Figure 24. Verify Jenkins service status.

Step 24 – Prepare the Jenkins/EC2 terminal environment

Instruction / Action: Use the Jenkins server terminal to prepare the CI/CD environment and verify access to the required project and AWS resources.

Expected Result: The Jenkins terminal environment is accessible and being prepared.



```
Verifying      : iptables-libs-1.8.8-3.amzn2023.0.2.x86_64      4/11
Verifying      : iptables-nft-1.8.8-3.amzn2023.0.2.x86_64      5/11
Verifying      : libgroup-3.0-1.amzn2023.0.1.x86_64            6/11
Verifying      : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64  7/11
Verifying      : libnftnl-1.0.1-19.amzn2023.0.2.x86_64        8/11
Verifying      : libnftnl-1.2.2-2.amzn2023.0.2.x86_64        9/11
Verifying      : pigz-2.5-1.amzn2023.0.4.x86_64              10/11
Verifying      : runc-1.3.5-1.amzn2023.0.2.x86_64             11/11

Installed:
container-selinux-4:2.245.0-1.amzn2023.noarch      containerd-2.2.5-1.amzn2023.0.2.x86_64      docker-25.0.16-1.amzn2023.0.4.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64      iptables-nft-1.8.8-3.amzn2023.0.2.x86_64      libgroup-3.0-1.amzn2023.0.1.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64  libnftnl-1.0.1-19.amzn2023.0.2.x86_64      libnftnl-1.2.2-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.4.x86_64      runc-1.3.5-1.amzn2023.0.2.x86_64

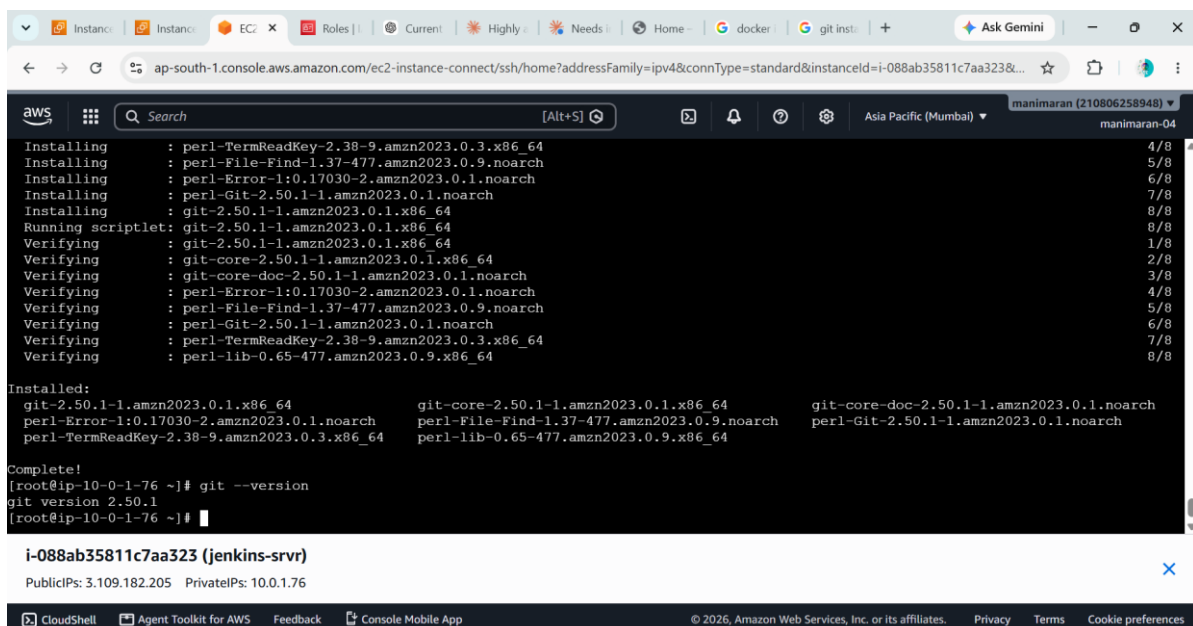
Complete!
[root@ip-10-0-1-76 ~]# sudo systemctl start docker
[root@ip-10-0-1-76 ~]# sudo systemctl enable docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[root@ip-10-0-1-76 ~]# sudo usermod -aG docker ec2-user
[root@ip-10-0-1-76 ~]# newgrp docker
[root@ip-10-0-1-76 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-10-0-1-76 ~]#
```

Figure 25. Prepare the Jenkins/EC2 terminal environment.

Step 25 – Execute/verify Jenkins terminal setup commands

Instruction / Action: Continue the Jenkins server setup and inspect the terminal output for successful configuration of the required tools and environment.

Expected Result: The required setup commands complete with usable terminal output.



```
Installing      : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64      4/8
Installing      : perl-File-Find-1.37-477.amzn2023.0.9.noarch     5/8
Installing      : perl-Error-1:0.17030-2.amzn2023.0.1.noarch     6/8
Installing      : perl-Git-2.50.1-1.amzn2023.0.1.noarch          7/8
Installing      : git-2.50.1-1.amzn2023.0.1.x86_64              8/8
Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64            8/8
Verifying      : git-2.50.1-1.amzn2023.0.1.x86_64              1/8
Verifying      : git-core-2.50.1-1.amzn2023.0.1.x86_64          2/8
Verifying      : git-core-doc-2.50.1-1.amzn2023.0.1.noarch      3/8
Verifying      : perl-Error-1:0.17030-2.amzn2023.0.1.noarch     4/8
Verifying      : perl-File-Find-1.37-477.amzn2023.0.9.noarch    5/8
Verifying      : perl-Git-2.50.1-1.amzn2023.0.1.noarch          6/8
Verifying      : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64     7/8
Verifying      : perl-lib-0.65-477.amzn2023.0.9.x86_64          8/8

Installed:
git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64      git-core-doc-2.50.1-1.amzn2023.0.1.noarch
perl-Error-1:0.17030-2.amzn2023.0.1.noarch  perl-File-Find-1.37-477.amzn2023.0.9.noarch  perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64  perl-lib-0.65-477.amzn2023.0.9.x86_64

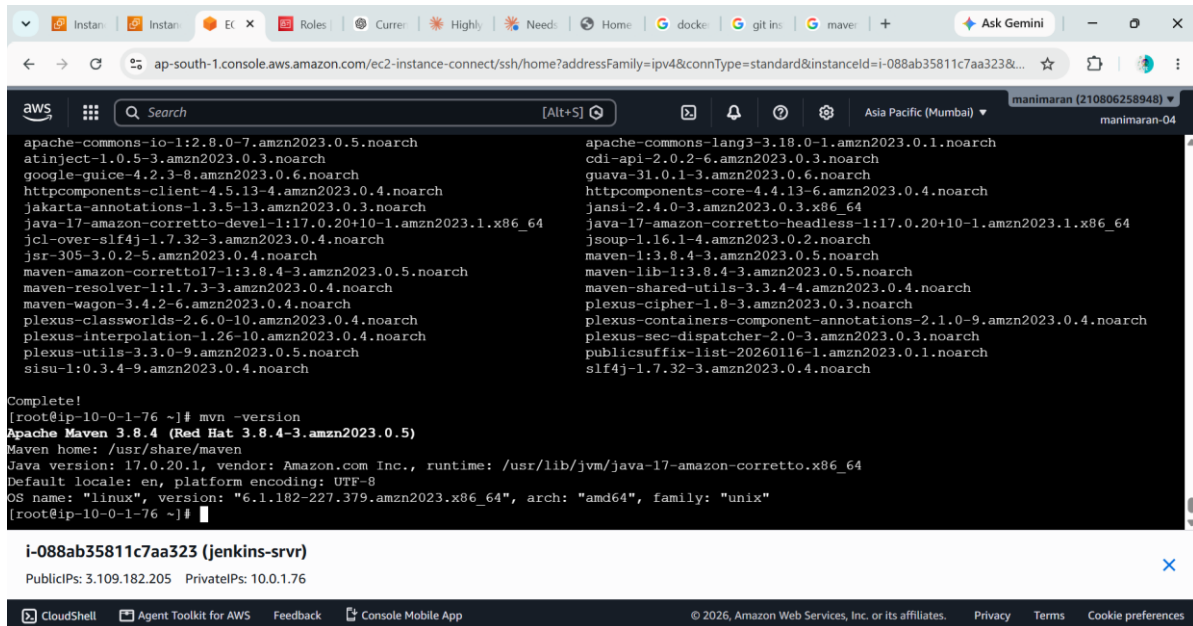
Complete!
[root@ip-10-0-1-76 ~]# git --version
git version 2.50.1
[root@ip-10-0-1-76 ~]#
```

Figure 26. Execute/verify Jenkins terminal setup commands.

Step 26 – Complete Jenkins installation/setup

Instruction / Action: Complete the remaining Jenkins/EC2 installation activity and verify the environment is ready for Jenkins administration and pipeline execution.

Expected Result: The Jenkins installation/setup activity is completed.



The screenshot shows the AWS CloudShell interface. The terminal window displays the output of the Jenkins installation process, including the installation of various Java dependencies and the completion of the Jenkins setup. The output shows the installation of Jenkins 2.400.3 on a Linux x86_64 architecture. Below the terminal output, the instance details for 'i-088ab35811c7aa323 (jenkins-srvr)' are shown, including public and private IP addresses.

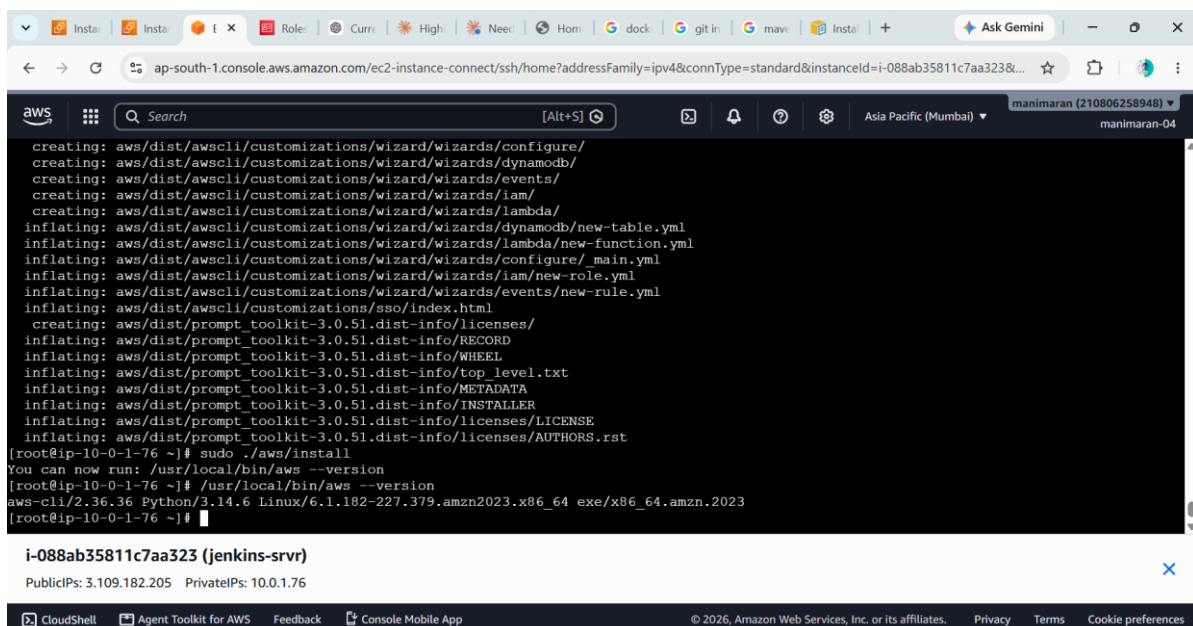
```
Complete!  
[root@ip-10-0-1-76 ~]# mvn -version  
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)  
Maven home: /usr/share/maven  
Java version: 17.0.20.1, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64  
Default locale: en, platform encoding: UTF-8  
OS name: "linux", version: "6.1.182-227.379.amzn2023.x86_64", arch: "amd64", family: "unix"  
[root@ip-10-0-1-76 ~]#  
  
i-088ab35811c7aa323 (jenkins-srvr)  
PublicIPs: 3.109.182.205 PrivateIPs: 10.0.1.76
```

Figure 27. Complete Jenkins installation/setup.

Step 27 – Verify the Jenkins/EC2 environment

Instruction / Action: Perform the final terminal verification of the Jenkins server environment before opening Jenkins in the browser.

Expected Result: The Jenkins server environment passes the final verification.



The screenshot shows the AWS CloudShell interface with the terminal window displaying the verification of the Jenkins environment. The output shows the installation of the Jenkins CLI and the verification of the Jenkins environment. The output shows the installation of the Jenkins CLI and the verification of the Jenkins environment.

```
creating: aws/dist/awsclicustomizations/wizard/wizards/configure/  
creating: aws/dist/awsclicustomizations/wizard/wizards/dynamodb/  
creating: aws/dist/awsclicustomizations/wizard/wizards/events/  
creating: aws/dist/awsclicustomizations/wizard/wizards/iam/  
creating: aws/dist/awsclicustomizations/wizard/wizards/lambda/  
inflating: aws/dist/awsclicustomizations/wizard/wizards/dynamodb/new-table.yml  
inflating: aws/dist/awsclicustomizations/wizard/wizards/lambda/new-function.yml  
inflating: aws/dist/awsclicustomizations/wizard/wizards/configure/main.yml  
inflating: aws/dist/awsclicustomizations/wizard/wizards/iam/new-role.yml  
inflating: aws/dist/awsclicustomizations/wizard/wizards/events/new-rule.yml  
inflating: aws/dist/awsclicustomizations/ssc/index.html  
creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE  
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst  
[root@ip-10-0-1-76 ~]# sudo ./aws/install  
You can now run: /usr/local/bin/aws --version  
[root@ip-10-0-1-76 ~]# /usr/local/bin/aws --version  
aws-cli/2.36.36 Python/3.14.6 Linux/6.1.182-227.379.amzn2023.x86_64 exe/x86_64.amzn.2023  
[root@ip-10-0-1-76 ~]#  
  
i-088ab35811c7aa323 (jenkins-srvr)  
PublicIPs: 3.109.182.205 PrivateIPs: 10.0.1.76
```

Figure 28. Verify the Jenkins/EC2 environment.

Step 28 – Unlock Jenkins

Instruction / Action: Open the Jenkins web interface using the Jenkins server endpoint. Retrieve the initial administrator password from the server-side location shown by Jenkins and complete the initial unlock/setup. Do not record the password in documentation.

Expected Result: Jenkins unlock/setup is available without exposing the administrator password.

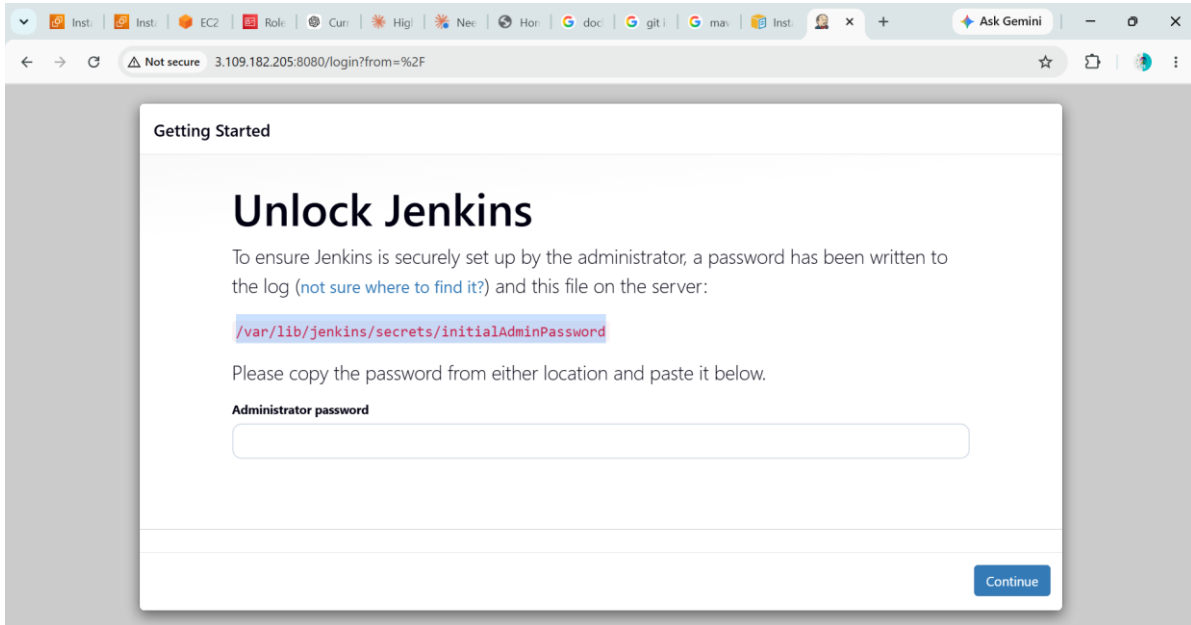


Figure 29. Unlock Jenkins.

Step 29 – Verify Jenkins plugins

Instruction / Action: Open Jenkins plugin management and verify the required GitHub, pipeline, credentials and integration plugins are installed/available for the CI/CD workflow.

Expected Result: Required Jenkins integrations are available.

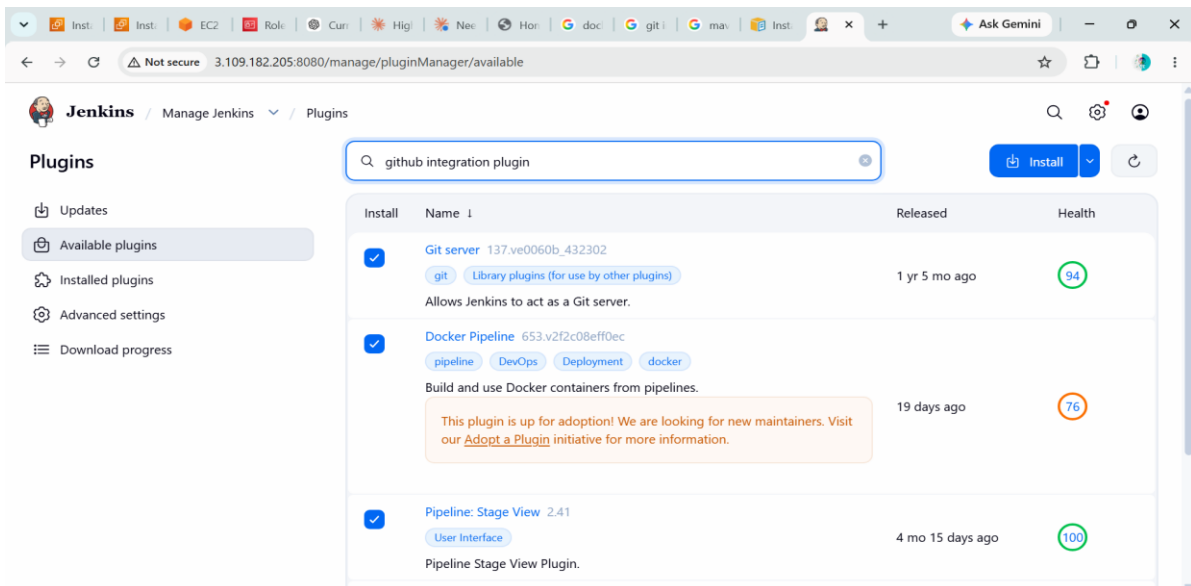


Figure 30. Verify Jenkins plugins.

Step 30 – Verify the GitHub repository

Instruction / Action: Open the GitHub repository and verify the application source and CI/CD files, including src/main, pom.xml, Dockerfile, Jenkinsfile and README.md.

Expected Result: The GitHub repository contains the required application and CI/CD files.

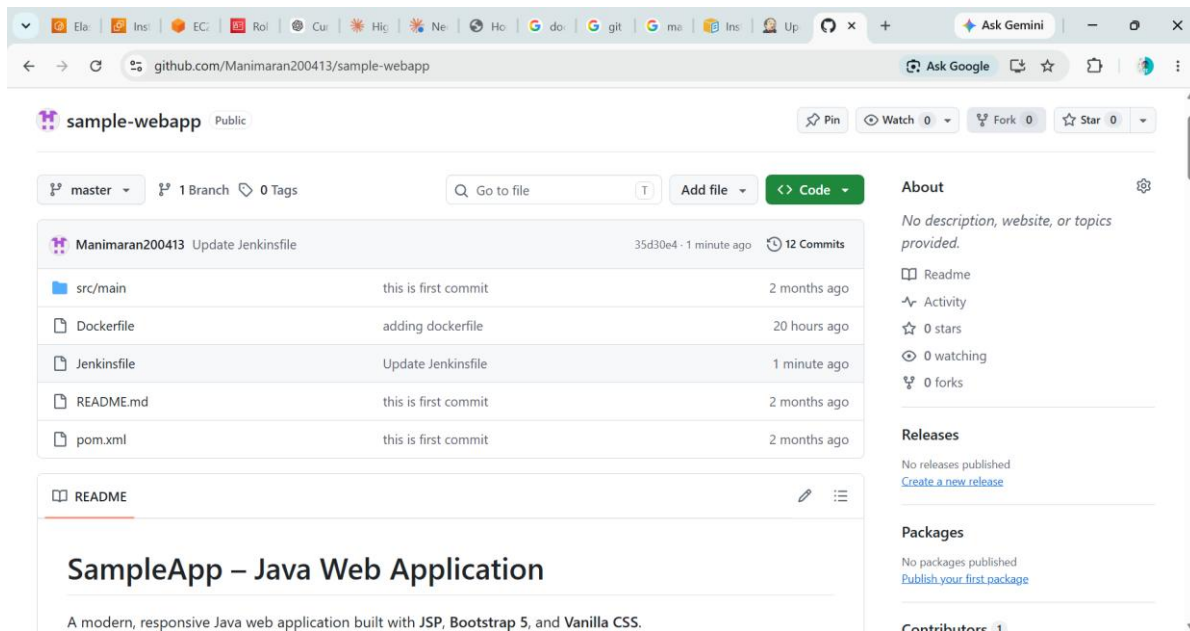


Figure 31. Verify the GitHub repository.

Step 31 – Configure/verify the GitHub webhook

Instruction / Action: Open the repository Webhooks settings and verify the Jenkins webhook configuration. Confirm the target URL/event configuration and investigate any failed delivery shown in the evidence.

Expected Result: The Jenkins webhook configuration is visible for repository events.

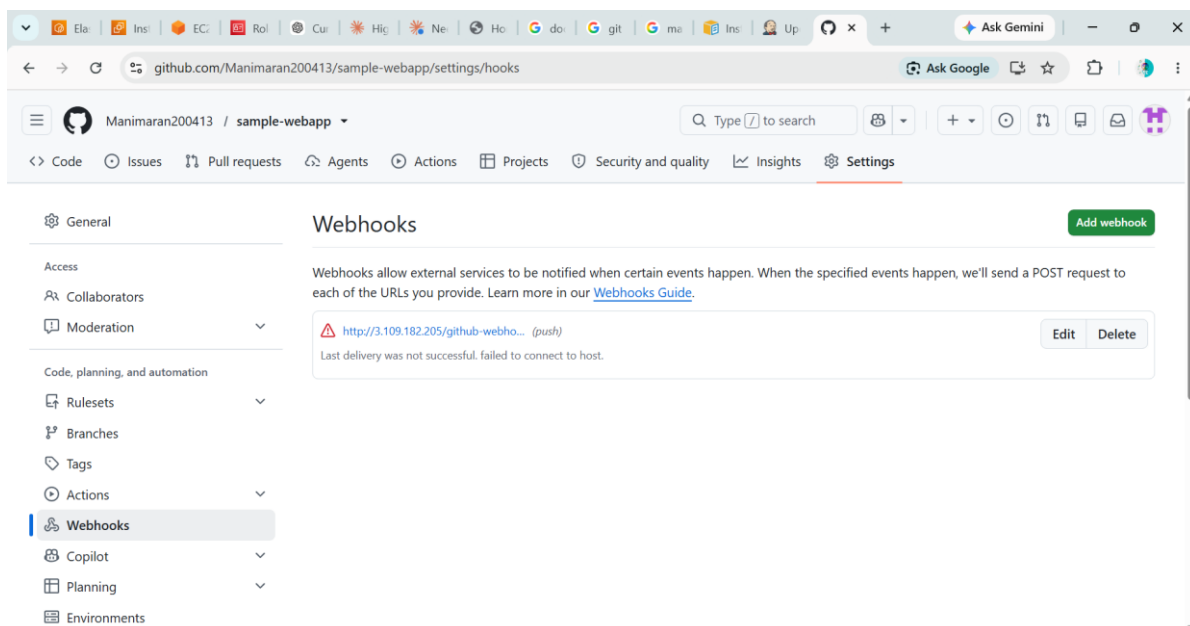


Figure 32. Configure/verify the GitHub webhook.

Step 32 – Verify the Jenkins pipeline project

Instruction / Action: Open the Jenkins pipeline job and confirm the project is configured and ready to execute the GitHub-to-Jenkins CI/CD workflow.

Expected Result: The Jenkins pipeline project is configured and ready.

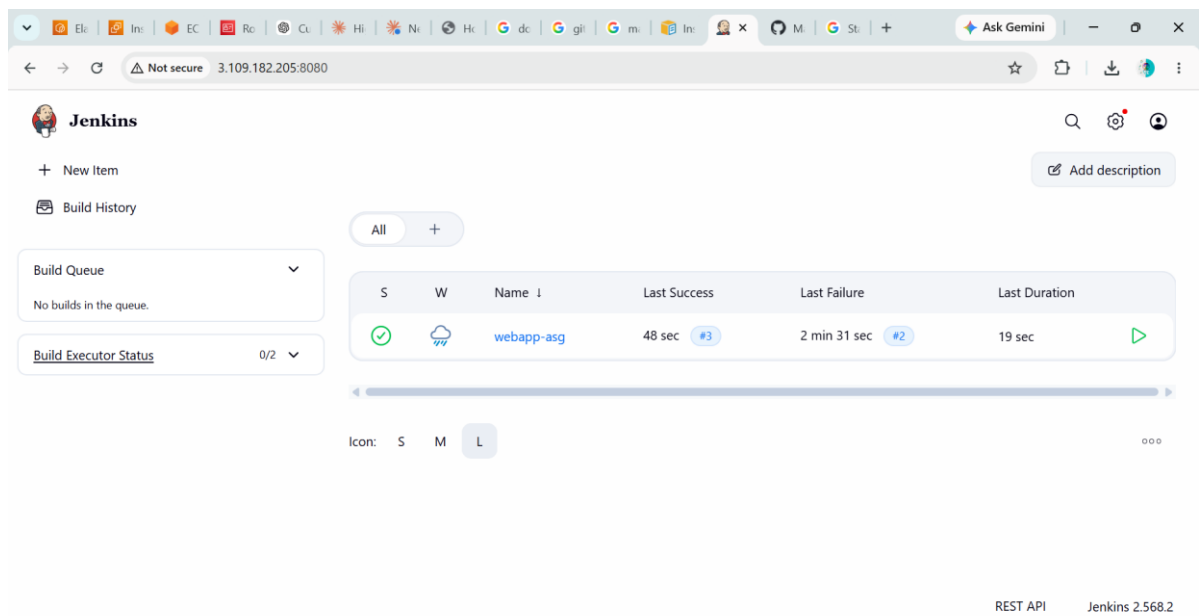


Figure 33. Verify the Jenkins pipeline project.

Step 33 – Run/verify Jenkins Stage View

Instruction / Action: Execute the pipeline and open the Stage View. Verify the checkout, build, Docker/ECR and deployment stages are displayed and the successful stages are shown in green.

Expected Result: Pipeline stages are visible and the successful run is shown.

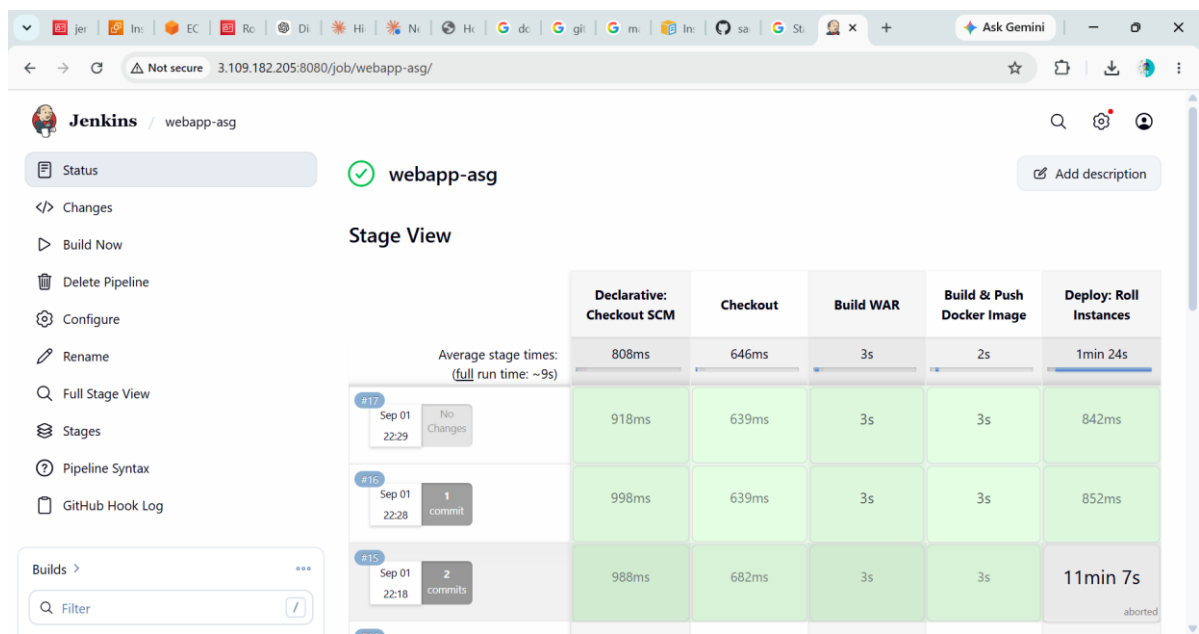
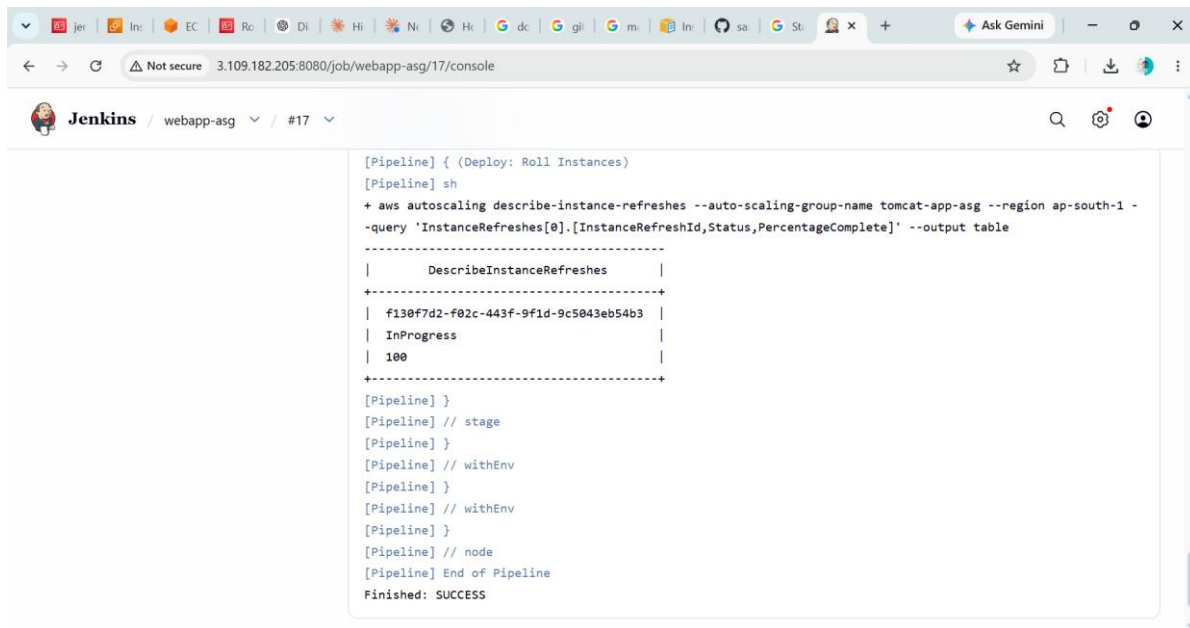


Figure 34. Run/verify Jenkins Stage View.

Step 34 – Verify deployment console output

Instruction / Action: Open the Jenkins console output and verify that the deployment stage invokes the Auto Scaling instance-refresh workflow. Confirm the refresh status reaches completion.

Expected Result: The deployment/instance-refresh commands and completion status are visible.



```
[Pipeline] { (Deploy: Roll Instances)
[Pipeline] sh
+ aws autoscaling describe-instance-refreshes --auto-scaling-group-name tomcat-app-asg --region ap-south-1 --
-query 'InstanceRefreshes[0].[InstanceRefreshId,Status,PercentageComplete]' --output table
-----
| DescribeInstanceRefreshes |
+-----+
| f130f7d2-f02c-443f-9f1d-9c5043eb54b3 |
| InProgress |
| 100 |
+-----+
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 35. Verify deployment console output.

Step 35 – Verify EC2/Auto Scaling deployment

Instruction / Action: Open AWS EC2/Auto Scaling and verify that the refreshed application instances are running and the deployment fleet reflects the updated application version.

Expected Result: The application EC2/Auto Scaling deployment is running as expected.

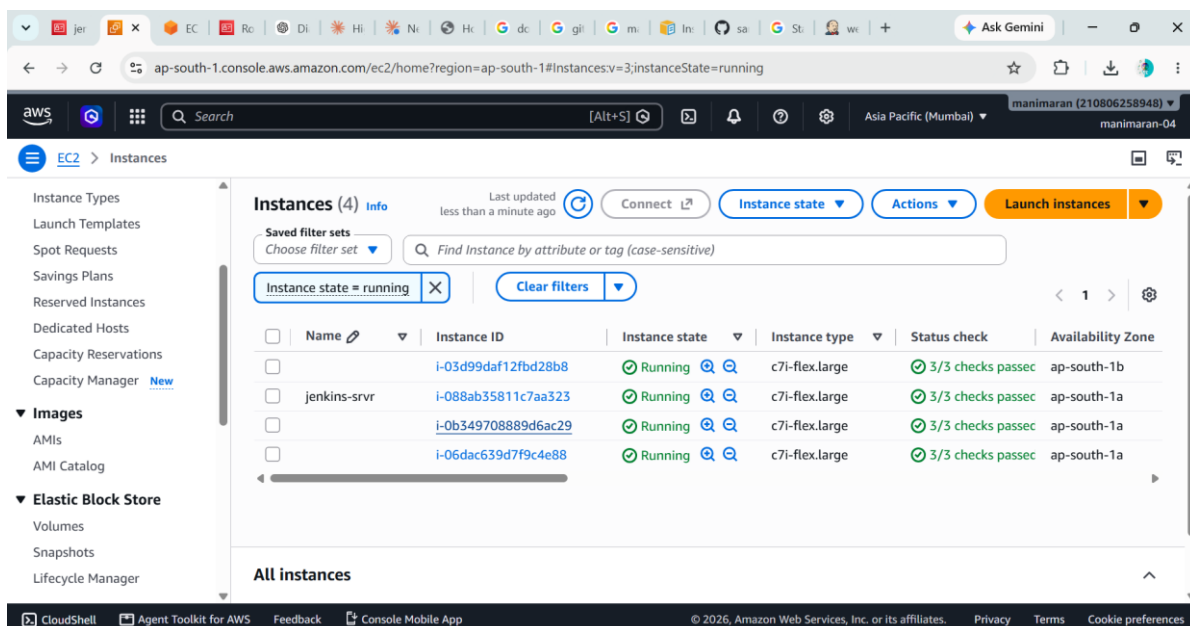


Figure 36. Verify EC2/Auto Scaling deployment.

Step 36 – Verify the ALB/application deployment

Instruction / Action: Open the load-balancing/application deployment view and confirm the ALB is active, the target group is associated, and the application is reachable through the public ALB endpoint.

Expected Result: The ALB and application deployment path are available.

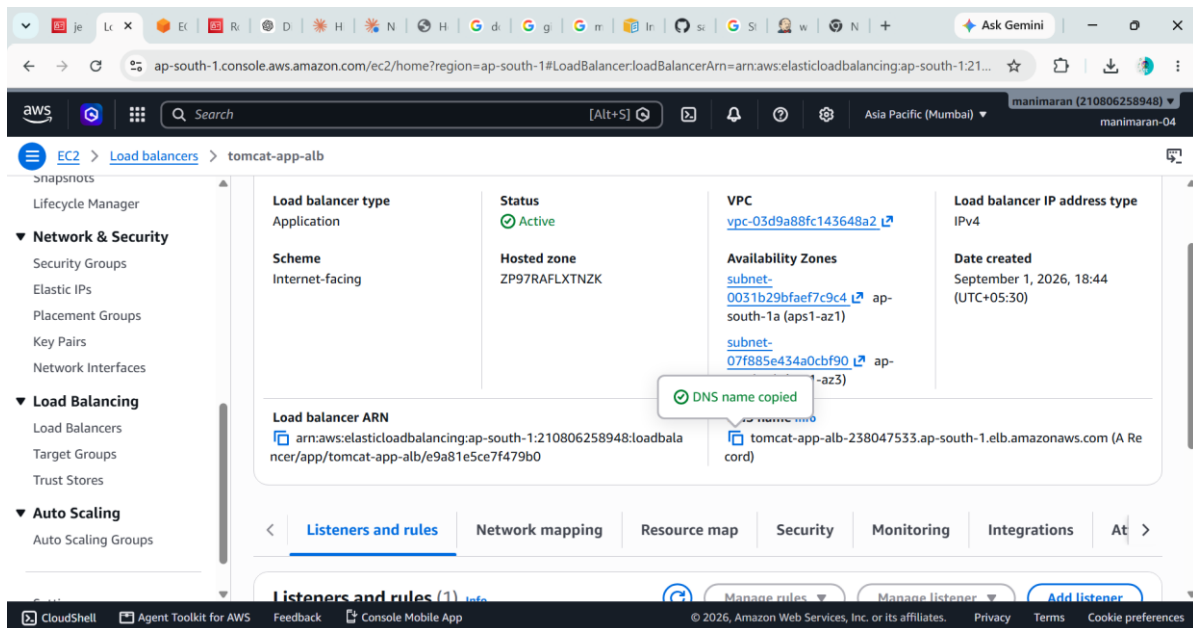


Figure 37. Verify the ALB/application deployment.

Step 37 – Validate the SampleApp Home page

Instruction / Action: Open the ALB DNS endpoint and verify the SampleApp Home page loads successfully. This validates the complete user-to-application request path.

Expected Result: The deployed SampleApp Home page loads successfully.

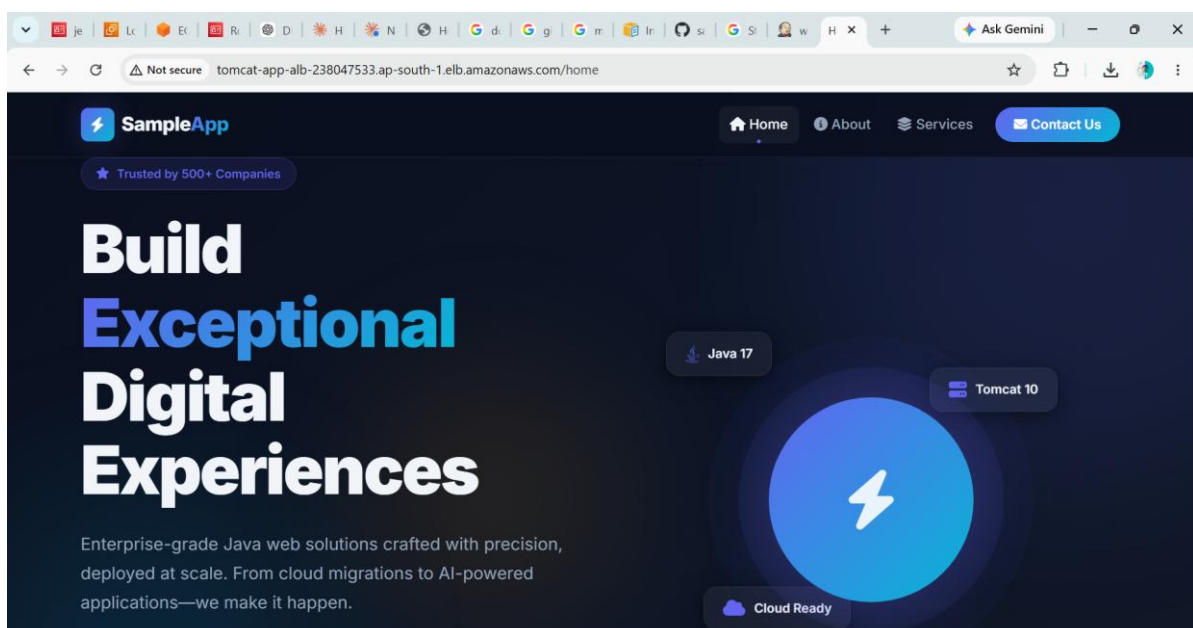


Figure 38. Validate the SampleApp Home page.

Step 38 – Validate the SampleApp Services page

Instruction / Action: Navigate to the Services page through the same ALB endpoint and verify that application routing and the deployed Java/Tomcat application continue to work.

Expected Result: The deployed SampleApp Services page loads successfully.

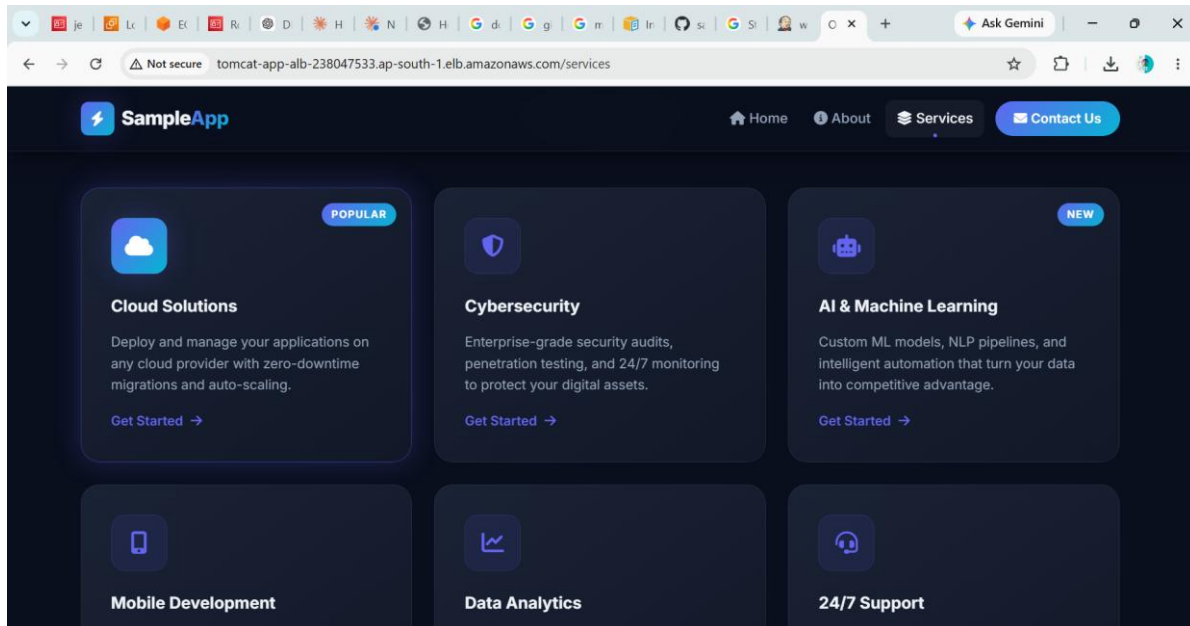


Figure 39. Validate the SampleApp Services page.

10. Jenkins CI/CD Pipeline

Declarative: Checkout SCM — Jenkins prepares the SCM checkout context.

Checkout — The application source is retrieved from GitHub.

Build WAR — Maven builds and packages the Java web application as a WAR file.

Build & Push Docker Image — Docker builds the application image and Jenkins pushes it to Amazon ECR.

Deploy: Roll Instances — Jenkins initiates an Auto Scaling instance refresh so the updated application version can roll through the fleet.

The Stage View shows the pipeline stages in green for the successful run. The deployment stage is the final transition from image publication to the refreshed application fleet.

Deployment verification: the Jenkins console evidence shows an AWS Auto Scaling describe-instance-refreshes operation returning PercentageComplete = 100, followed by "Finished: SUCCESS".

11. Jenkins Server Environment

- Java 17 was available on the Jenkins server.
- Apache Maven 3.8.4 was installed and verified.
- Git 2.50.1 was installed and verified.
- Docker 25.x was installed and the Docker service was started/enabled.
- Jenkins was enabled and running as a systemd service.
- AWS CLI was installed and verified.

12. AWS Infrastructure

12.1 Network Layer

- Amazon VPC provides the isolated network boundary.
- Public and private subnet resources are used as shown in the supplied infrastructure evidence.
- An Internet Gateway provides Internet connectivity for public-facing components.
- A NAT Gateway provides outbound connectivity for resources that are not directly Internet-facing.
- Route tables control traffic paths between the VPC subnets and gateways.

12.2 Load Balancing Layer

- The Application Load Balancer is internet-facing.
- The ALB receives HTTP requests on port 80.
- The target group contains the application targets.
- Application traffic is forwarded toward Tomcat on port 8080.

12.3 Application Compute Layer

- EC2 instances provide the application compute capacity.
- Docker runs the application container on each application instance.
- Tomcat 10 serves the Java web application on port 8080.
- The Auto Scaling Group manages the application instance fleet.
- The launch template standardizes instance initialization.

13. IAM and Amazon ECR Integration

The supplied evidence shows an IAM role named "jenkins-ecr-asg-role" associated with an EC2 environment. This role is used as the AWS identity for EC2-side operations that require AWS permissions, including the application image retrieval workflow. Amazon ECR stores the Docker image produced by the Jenkins pipeline.

14. Application Deployment and Validation

- The deployed application is the SampleApp Java web application.
- The Home page is reachable through the ALB endpoint.
- The application displays SampleApp branding and Java/Tomcat references.
- The Services page is reachable through the same ALB endpoint.
- The demonstrated runtime path is ALB → target group → EC2 → Docker → Tomcat 10 → Java web application.

15. Application Request Flow

User → Application Load Balancer :80 → Target Group → EC2 → Docker → Tomcat 10 :8080 → Java Web Application

16. Rolling Deployment Flow

1. Developer pushes a new application version to GitHub.
2. Jenkins checks out the updated source.
3. Maven generates the updated WAR artifact.
4. Docker builds the new application image.
5. Jenkins pushes the image to Amazon ECR.
6. Jenkins initiates an Auto Scaling Group instance refresh.
7. Instances are rolled/refreshed according to the Auto Scaling configuration.
8. Refreshed application instances run the updated Docker/Tomcat application.
9. The ALB continues routing user traffic through the target group.
10. Jenkins verifies the refresh state and completes the build with SUCCESS when the refresh reaches completion.

17. Verification Checklist

- ✓ GitHub repository contains the Java application and CI/CD files.
- ✓ Jenkins is installed and running.
- ✓ Java 17 is available.
- ✓ Maven is installed and verified.
- ✓ Git is installed and verified.
- ✓ Docker is installed and running.
- ✓ AWS CLI is installed and verified.
- ✓ VPC networking resources are present.
- ✓ Security groups are configured.
- ✓ Launch template is available.
- ✓ Target group is configured.
- ✓ Application Load Balancer is active.
- ✓ Auto Scaling Group is available.
- ✓ Application EC2 instances are running.
- ✓ Amazon ECR repository/image is available.
- ✓ SampleApp is reachable through the ALB DNS name.
- ✓ Jenkins pipeline stages complete successfully.
- ✓ Instance refresh reaches 100 percent completion in the demonstrated run.

18. Troubleshooting and Operational Notes

GitHub webhook connectivity: The supplied GitHub Webhooks evidence shows a configured webhook, but the displayed last delivery was not successful and reported a connection failure. Treat this as a webhook-connectivity issue to investigate separately from the successful Jenkins build evidence.

Jenkins first-start security: The Jenkins unlock page shows the initial administrator password file location. The password itself is not reproduced in this report.

ECR image naming: Check the Docker/ECR repository name and image-tag consistency before pushing or deploying an image.

ALB target health: If the application is unavailable through the ALB, verify that the target group contains healthy targets, Tomcat is listening on port 8080, and the relevant security-group rules allow ALB-to-application traffic.

ECR pull failures: If an application instance cannot retrieve the image, verify EC2 IAM role permissions, the ECR repository URI, AWS region and the Docker/ECR authentication sequence.

Rolling deployment: During an instance refresh, temporary changes in instance count or health can occur. Confirm refresh status and target health before considering deployment complete.

19. Security Notes

- Do not commit AWS access keys, secret keys, passwords, tokens or private-key material to GitHub.
- Treat the Jenkins initial administrator password as sensitive and do not include it in documentation.
- Prefer IAM roles/instance profiles over hard-coded long-lived AWS credentials on EC2 hosts.
- Expose only the ports required for the intended traffic flow in security groups.
- Use the public ALB as the normal Internet entry point and avoid unnecessary direct exposure of application instances.

20. Key Lessons Learned

- Jenkins can automate the complete application delivery lifecycle from source checkout through deployment.
- Maven provides a repeatable Java build and WAR packaging process.
- Docker creates a consistent runtime package for the Java/Tomcat application.
- Amazon ECR provides a managed private registry for Docker images.
- An Application Load Balancer separates the public entry point from individual application instances.
- Auto Scaling provides application capacity management and supports instance refresh for rolling deployment.
- IAM instance roles reduce the need to store long-lived AWS credentials on EC2 hosts.
- A Jenkinsfile makes the CI/CD workflow explicit and repeatable.
- Pipeline stage visualization helps verify where a deployment succeeded or failed.
- Application validation through the ALB is an important end-to-end verification step.

21. Interview / Viva Explanation

I built an AWS CI/CD project for a Java web application. The source code is stored in GitHub and Jenkins automates the delivery pipeline. Jenkins checks out the code, uses Maven to build the WAR file, creates a Docker image and pushes it to Amazon ECR. The application runs in Docker on Tomcat 10 on EC2 instances managed by an Auto Scaling Group. Users access the application through an Application Load Balancer on port 80, and the ALB forwards requests to the application targets on port 8080. For deployment, Jenkins starts an Auto Scaling instance refresh so application instances can be rolled to the updated version. The demonstrated successful run shows the refresh reaching 100 percent and Jenkins finishing with SUCCESS.

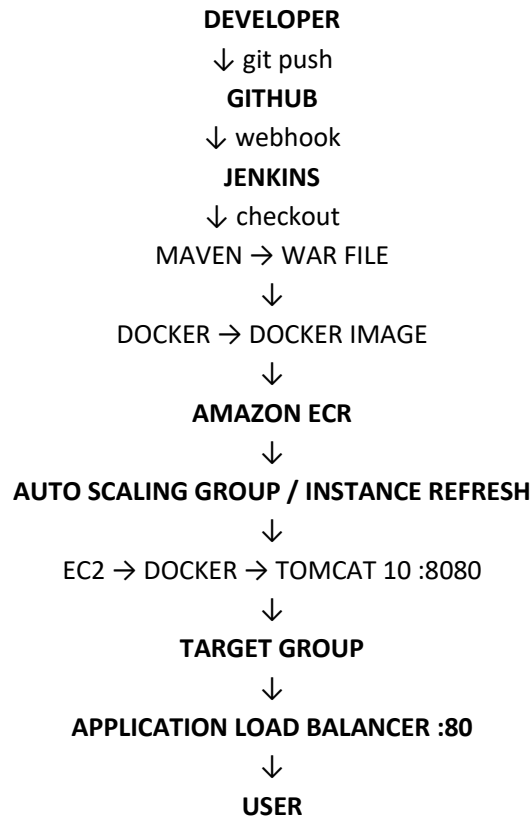
22. Conclusion

The Level 6 project demonstrates a practical AWS CI/CD implementation for a Java web application. The workflow connects GitHub source control with Jenkins automation, Maven build and WAR packaging, Docker containerization, Amazon ECR image storage, EC2 Auto Scaling deployment, and Application Load Balancer based application access. The evidence demonstrates AWS resource configuration, Jenkins administration, the SampleApp browser interface and a successful rolling instance refresh.

The complete delivery path,

GitHub → Jenkins → Maven → WAR → Docker → ECR → Auto Scaling/EC2 → Tomcat 10 → Target Group → ALB → User.

Appendix A – Complete CI/CD Flow



Appendix B – Screenshots Index

- Screenshot 1: Step 1 – Create/verify the VPC — Figure 2.
- Screenshot 2: Step 2 – Configure the subnets — Figure 3.
- Screenshot 3: Step 3 – Attach/configure the Internet Gateway — Figure 4.
- Screenshot 4: Step 4 – Create/verify the NAT Gateway — Figure 5.
- Screenshot 5: Step 5 – Configure the route tables — Figure 6.
- Screenshot 6: Step 6 – Verify the ECR image/repository — Figure 7.
- Screenshot 7: Step 7 – Configure the EC2 security group — Figure 8.
- Screenshot 8: Step 8 – Verify additional application security-group details — Figure 9.
- Screenshot 9: Step 9 – Create/configure the launch template — Figure 10.
- Screenshot 10: Step 10 – Verify the launch template — Figure 11.
- Screenshot 11: Step 11 – Create/configure the target group — Figure 12.
- Screenshot 12: Step 12 – Create/configure the Application Load Balancer — Figure 13.
- Screenshot 13: Step 13 – Start Auto Scaling Group creation — Figure 14.
- Screenshot 14: Step 14 – Select availability zones/subnets — Figure 15.
- Screenshot 15: Step 15 – Configure Auto Scaling Group capacity — Figure 16.
- Screenshot 16: Step 16 – Finalize the Auto Scaling Group — Figure 17.

Screenshot 17: Step 17 – Verify the Auto Scaling Group — Figure 18.

Screenshot 18: Step 18 – Verify running EC2 instances — Figure 19.

Screenshot 19: Step 19 – Validate the SampleApp through the ALB — Figure 20.

Screenshot 20: Step 20 – Review IAM policies — Figure 21.

Screenshot 21: Step 21 – Verify IAM roles — Figure 22.

Screenshot 22: Step 22 – Verify the EC2 Jenkins/application environment — Figure 23.

Screenshot 23: Step 23 – Verify Jenkins service status — Figure 24.

Screenshot 24: Step 24 – Prepare the Jenkins/EC2 terminal environment — Figure 25.

Screenshot 25: Step 25 – Execute/verify Jenkins terminal setup commands — Figure 26.

Screenshot 26: Step 26 – Complete Jenkins installation/setup — Figure 27.

Screenshot 27: Step 27 – Verify the Jenkins/EC2 environment — Figure 28.

Screenshot 28: Step 28 – Unlock Jenkins — Figure 29.

Screenshot 29: Step 29 – Verify Jenkins plugins — Figure 30.

Screenshot 30: Step 30 – Verify the GitHub repository — Figure 31.

Screenshot 31: Step 31 – Configure/verify the GitHub webhook — Figure 32.

Screenshot 32: Step 32 – Verify the Jenkins pipeline project — Figure 33.

Screenshot 33: Step 33 – Run/verify Jenkins Stage View — Figure 34.

Screenshot 34: Step 34 – Verify deployment console output — Figure 35.

Screenshot 35: Step 35 – Verify EC2/Auto Scaling deployment — Figure 36.

Screenshot 36: Step 36 – Verify the ALB/application deployment — Figure 37.

Screenshot 37: Step 37 – Validate the SampleApp Home page — Figure 38.

Screenshot 38: Step 38 – Validate the SampleApp Services page — Figure 39.